Tampere University

Miika Kuusisto

HTTP HEADER FIELDS AND THEIR VERSATILE USE

Master's thesis
Faculty of Information Technology and Communication Sciences
Juha Vihervaara
Mikko Valkama
September 2024

ABSTRACT

Miika Kuusisto: HTTP Header fields and their versatile use
Master's thesis
Tampere University
Master's Degree Programme in Information Technology
September 2024

This research goes over how HTTP headers work as a part of the HTTP communication pipeline and how HTTP has evolved over time and the headers alongside it from HTTP/1.1 to HTTP/2 and HTTP/3. Then this study then delves deeper into how the HTTP works and how it was designed from client interactions to server and vice versa. The study also goes over the introduction of QUIC in HTTP/3 that replaces previously used TCP connections. QUIC is then compared to TCP both in terms of features and performance.

A sizeable portion of the study is dedicated to introducing HTTP headers from their design on how and why they are used. Then the general categories of headers are explained, as every header belongs to one of them. Afterwards specific categories for headers are given with example headers for each of them, giving a look into the variety of situations and use cases for header fields. Also mentioned in the study are additional header field types that don't fit a normal use case, ones like experimental headers and non-standard headers. The study also includes examples of captured packets for HTTP/1.1, HTTP/2 and HTTP/3 and showcases the unique aspects of them as well as the general use of headers in them. For each version of HTTP, there are packets captured from the client side for both occasions, when sending packets to a server and when receiving packets as an answer from the server.

The study also includes a more technical look into the compression algorithms, HPACK introduced in HTTP/2 and QPACK introduced in HTTP/3. The algorithms are first introduced and looked at on a deeper level as well as discussing possible security risks of their implementations. Lastly this study compiles data from multiple other studies on efficiency and performance of compression algorithms and the different versions of HTTP as well and gives a general look into HPACK's performance and efficiency.

In this study it was found that multiple researches concluded that in general the performance of each version of HTTP improves when compared to the previous one. Yet in the many studies gathered the favoured version of HTTP varied, this was concluded to be due to each version of HTTP having its own strengths and weaknesses and not obsoleting each other. For compression algorithms research was found for HPACK but not for QPACK, this was thought to be due to the comparatively recent appearance of QPACK. In the notes of one similar study it was also mentioned that no publicly available studies for QPACK were found. Thus, it was concluded that for now the compression algorithms could not be compared in a more direct way and this would be up to future research. HPACK itself had been tested and was found to have implementations that performed much better than naive implementations.

Keywords: HTTP, HEADER, TCP, QUIC, Compression Algorithm, HPACK, QPACK

TIIVISTELMÄ

Miika Kuusisto: HTTP otsaketiedot ja niiden monipuolinen käyttö
Diplomityö
Tampereen yliopisto
Tietotekniikan DI-tutkinto-ohjelma
Syyskuu 2024

Tässä tutkimuksessa käydään läpi HTTP protokollaa ja erityisesti käsitellään sen otsaketietoja, jotka ovat tärkeä osa HTTP kommunikaatio protokollaa. Työssä käydään myös läpi, kuinka HTTP protokolla ja otsaketiedot ovat kehittyneet versioiden myötä alkaen HTTP/1.1 ja HTTP/2 loppuen uusimpaan HTTP/3 versioon. HTTP:n toimintaa tutkitaan myös syvemmin ja katsotaan, kuinka se suunniteltiin sekä käyttäjän näkökulmasta että palvelimen puolesta. Työssä käsitellään myös HTTP/3 versiossa esiintyvää QUIC protokollaa, jonka tehtävänä on korvata aikaisemmissa versioissa käytetty TCP protokolla. QUIC ja TCP protokollien ominaisuuksia käydään läpi ja tutkitaan niiden eroja sekä protokollien suorituskykyä.

Merkittävä osa työstä käytetään HTTP otsaketietojen läpikäymiseen, kuinka otsaketiedot suunniteltiin ja miten sekä miksi niitä käytetään. Yleiset kategoriat, joista vähintään yhteen jokainen otsaketieto kuuluu, käydään läpi. Tämän jälkeen käydään läpi yksityiskohtaisemmin eri käyttökohteita ja kategorioita otsaketiedoille ja annetaan esimerkkejä näihin kuuluvista otsaketiedoista. Työssä myös käydään läpi tavallisista kategorioista eroavia otsaketietoja kuten kokeelliset otsaketiedot ja ei standardiin kuuluvat otsaketiedot. Työhön kuuluu myös esimerkkejä kaapatuista HTTP paketeista HTTP/1.1, HTTP/2 sekä HTTP/3 versioille, näille versioille on esimerkkejä sekä käyttäjän lähettämistä viesteistä että lähetettyjen viestien vastauksista. Esimerkeissä käydään läpi kaappauksien erikoisuuksia ja eroavuuksia versioiden välillä.

Lopuksi työssä käydään läpi otsaketietojen kompressointi algoritmeja. HPACK, joka suunniteltiin HTTP/2 versiossa ja QPACK, joka suunniteltiin HTTP/3 versiossa. Molemmat algoritmit käydään läpi perehtyen hyvin niiden suunnitteluun ja toteutukseen sekä käydään läpi mahdolliset turvallisuus riskit. Viimeisessä osiossa työtä on kooste useammasta tutkimuksesta, joissa käydään läpi HTTP versioiden välisiä tehokkuuksia tai kompressointi algoritmin toteutuksen tehokkuutta. Tutkimuksien tuloksia käydään läpi ja muodostetaan johtopäätöksiä näiden samanlaisuuksista tai eroavuuksista.

Tässä työssä havaittiin useamman kerätyn tutkimuksen avulla, että yleisesti jokainen HTTP versio toimii tehokkaammin kuin edellinen. Kerätyissä tutkimuksissa löytyi myös tuloksia, jotka suosivat vanhempia versioita, tämä pääteltiin johtuvan HTTP versioiden eriävistä vahvuuksista ja heikkouksista. Kompressointi algoritmeista löytyi tehtyjä testejä HPACK:lle mutta ei QPACK:lle, tämä ajateltiin johtuvan QPACK:n suhteellisesta uutuudesta. Yhdessä samanlaisessa tutkimuksessa myös mainittiin, ettei julkisia testejä löytynyt QPACK:lle. Täten todettiin, ettei kompressointi algoritmeja pystytä tässä työssä vertaamaan suoraan toisiinsa ja tämä jäisi odottamaan tulevaisuuden tutkimuksia. Kerätyissä HPACK testeissä havaittiin toteutuksia, jotka toimivat huomattavasti tehokkaammin kuin naiivi toteutukset.

Avainsanat: HTTP, Otsaketieto, TCP, QUIC, Kompressointi Algoritmi, HPACK, QPACK

CONTENTS

1. INTRODUCTION 1
2. HTTP BACKGROUND 2
    2.1 How HTTP came to be 2
    2.2 HTTP design 3
    2.3 QUIC vs TCP 7
3. HTTP HEADER FIELDS 11
    3.1 In depth look at header fields 12
    3.2 Primary header field categories 12
    3.3 Normal categories of header fields 13
    3.4 Additional categories of header fields 19
    3.5 Captured HTTP packets 20
4. COMPRESSION ALGORITHMS AND PERFORMANCE 26
    4.1 Compression algorithms 26
    4.2 HPACK 27
    4.3 QPACK 35
    4.4 Performance and results 52
5. CONCLUSIONS 55
REFERENCES 56

LIST OF FIGURES

Figure 1: HTTP/1.1 Packet sent from client to server 20
Figure 2: HTTP/1.1 Packet from the server to the client 21
Figure 3: HTTP/2 packet from client to server 22
Figure 4: HTTP/2 packet sent from server to client 23
Figure 5: HTTP/2 packet showing payload data 24
Figure 6: HTTP/3 packet sent from client to server 24
Figure 7: HTTP/3 packet sent from server to client 25

LIST OF SYMBOLS AND ABBREVIATIONS

CRIME	Compression Ratio Info-leak Made Easy
EOS	End-of-String
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IETF	Internet Engineering Task Force
MIME	Multipurpose Internet Mail Extension
MSS	Maximum Segment Size
QUIC	Quick UDP Internet Connections
RFC	Request for Comments
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
URL	Uniform Resource Locator
WWW	World Wide Web

.

1. INTRODUCTION

HTTP (Hypertext Transfer Protocol) is very widely used as it is the underlying mechanism used throughout the internet, and every website needs HTTP itself to function. Because HTTP is the framework for the internet it is well known on a base level, but perhaps less known is how HTTP works itself and the headers that are a crucial part of HTTP communication. The purpose of the headers is to provide the user requesting the website effectively the best version of the site the server can provide, including options like language, format and size. These headers also have evolved with HTTP and have become very versatile in what they do.

This research is focused on the HTTP headers and how they are used. Since there are many types of headers, the general categories will be mentioned along with specific categories of headers and some examples for them. This research also investigates the compression algorithms used on headers, HPACK header compression featured in HTTP/2 and QPACK a variation of HPACK that is introduced in HTTP/3 and their effects. These compression algorithms are also compared by their features, security risks and efficiency. Lastly are gathered results from other studies on the performance of different HTTP versions and the compression algorithms.

2. HTTP BACKGROUND

This section covers the necessary parts of HTTP to understand and follow later chapters on how the header fields work. This covers the formation of HTTP and its design and at the end further discusses a major change in the transport layer when HTTP/3 was introduced.

2.1 How HTTP came to be

HTTP was created for the WWW (World Wide Web) architecture and has evolved over time to support the needs of a worldwide hypertext system. HTTP is a family of stateless, application-level, request/response protocols that share a generic interface, extensible semantics, and self-descriptive messages. These attributes of HTTP enable flexible interaction with network-based hypertext information systems.

HTTP originally introduced in 1990 has been the primary information transfer protocol for the World Wide Web. It began as a simple mechanism for low-latency requests, with a single method (GET) to request transfer of a hypertext document identified by a given pathname. Afterwards HTTP was extended to enclose requests and responses within messages, transfer arbitrary data formats using MIME[1] (Multipurpose Internet Mail Extension-like) media types and route requests through intermediaries. These protocols were eventually defined as HTTP/0.9 and HTTP/1.0.[2] HTTP/1.0 which was the first release with a version number has documentation on the standard itself found at [3]. HTTP uses a "<major>.<minor>" numbering scheme to indicate versions of the protocol. The protocol versioning policy is intended to allow the sender to indicate the format of a message and its capacity for understanding further HTTP communication, rather than the features obtained via that communication. [4]

HTTP/1.1 documented in RFC (Request For Comments) 2616 [5] was designed to refine the protocol's features while retaining compatibility with the existing text-based messaging syntax. These designs were intended to improve HTTP interoperability, scalability, and robustness across the internet. This included length-based data delimiters for both fixed and dynamic (chunked) content, a consistent framework for content negotiation and opaque validators for conditional requests. Also designs like cache controls for better cache consistency, range requests for partial updates, and default persistent connections were included. HTTP/2 as described in RFC 7540 [6] introduced a multiplexed session layer on top of the existing TLS (Transport Layer Security) and TCP (Transmission

Control Protocol) protocols for exchanging concurrent HTTP messages with efficient field compression and server push.

TLS [7] consists of two primary components, a handshake protocol and a record protocol. The handshake protocol authenticates the communicating parties, negotiates cryptographic modes and parameters, and establishes shared keying material. The handshake protocol is designed to resist tampering, an active attacker should not be able to force the peers to negotiate different parameters than they would if the connection were not under attack. The record protocol uses the parameters established by the handshake protocol to protect traffic between the communicating peers. The record protocol divides traffic up into a series of records, each of which is independently protected using the traffic keys. [8]

TCP provides a reliable, in-order, byte stream service to applications. The byte-stream is conveyed over the network via TCP segments, with each TCP segment sent as an Internet Protocol datagram[9]. TCP reliability consists of detecting packet losses and errors as well as correction via retransmission. Data flow is supported bidirectionally over TCP connections, though applications are free to send data only unidirectionally, if they so choose. [10]

HTTP/3 described in RFC 9144 [11] provides greater independence for concurrent messages by using QUIC (Quick UDP Internet Connections) as a secure multiplexed transport over UDP (User Datagram Protocol) instead of TCP. QUIC documentation is described in the RFC 9000 [12]. The differences between TCP and QUIC are discussed in detail in chapter 2.3.

The HTTP versions used today include HTTP/1.1, HTTP/2 and HTTP/3. HTTP versions have not obsoleted each other because each one has specific benefits and limitations depending on how they are used. Implementations are then expected to choose the most appropriate transport and messaging syntax for their own use case.

2.2 HTTP design

The typical flow of data involves a client machine making a request to a server which then sends a response message. These HTTP requests are a way for things like web browsers to ask for information they need to load the website. An HTTP request always starts with a start line, followed by a block of headers and then, optionally, followed by an HTTP body.[13] For a request header, the start line consists of a method (GET, HEAD, POST etc.), a request URL (Uniform Resource Locator), and a version.[13]

The URL is simply the address of the wanted resource on the computer network. The HTTP method indicates the action the client wants from the server, commonly it is either the 'GET' or 'POST' methods. The 'GET' method request calls for information from the server to the client, this is usually the wanted website. The 'POST' method is the opposite of the 'GET' method in that it usually indicates that the client is submitting information to the server, for example information used to log-in such as username and password. The HTTP request's optional body is used to send any information that needs to be submitted to the server such as the 'POST' method's information. Lastly is the HTTP request header, which contains in key-value pairs the core information such as what browser is used and what data is being requested.

For the server its HTTP response contains a HTTP status code, HTTP response headers and the optional HTTP body. The status code is a 3-digit code that indicates whether the request was successful or what possible problem has happened. In the response body is the information that was requested by the client for example the HTML (Hypertext Markup Language) data, which the web browser uses to load the webpage. Lastly the HTTP response headers much like the request headers include key-value pairs of important information such as the language and format of the data being sent.

The details of this service that HTTP provides is hid by presenting a uniform interface to clients that is independent of the types of resources provided. Likewise, servers do not need to be aware of each client's purpose. A client request can be considered in isolation rather than being associated with a specific type of client or a predetermined sequence of application steps. This allows for general-purpose implementations to be used effectively in many different contexts. It also reduces interaction complexity and enables independent evolution over time.

HTTP is also designed for use as an intermediation protocol, where proxies and gateways can translate non-HTTP information systems into a more generic interface. These proxies and gateways include devices used to deliver the HTTP message from the client to server or vice versa that are not the server or the client itself.

One consequence of this flexibility is that the HTTP protocol cannot be defined in terms of what occurs behind the interface. Instead it is limited to defining the syntax of communication, the intent of received communication and the expected behaviour of recipients. If the communication is considered in isolation, then successful actions ought to be reflected in corresponding changes to the observable interface provided by servers. However, since multiple clients might act in parallel and perhaps at cross-purposes, we cannot require that such changes be observable beyond the scope of a single response. [2]

So, a client constructs request messages that communicate its intentions and routes those messages toward an identified origin server. A server listens for requests, parses each message received, interprets the message semantics in relation to the identified target resource, and responds to that request with one or more response messages. The client then examines the received responses to see if its intentions were carried out, determining what to do next based on the status codes and content received. The target of an HTTP request is called a "resource". HTTP does not limit the nature of a resource, it merely defines an interface that might be used to interact with resources. Most resources are identified by a URI (Uniform Resource Identifier).

For HTTP semantics, they include the intentions defined by each request method and extensions to those semantics that might be described in request header fields. Included are also status codes that describe the response and other control data and resource metadata that might be given in response fields. Semantics also include representation metadata that describe how content is intended to be interpreted by a recipient, request header fields that might influence content selection, and the various selection algorithms that are collectively referred to as "content negotiation".[2]

One design goal of HTTP is to separate resource identification from request semantics, which is made possible by vesting the request semantics in the request method and a few request-modifying header fields. A resource cannot treat a request in a manner inconsistent with the semantics of the method of the request. For example, though the URI of a resource might imply semantics that are not safe, a client can expect the resource to avoid actions that are unsafe when processing a request with a safe method.[2]

As mentioned before HTTP's version number consists of two decimal digits separated by a "." (period or decimal point). The first digit (major version) indicates the messaging syntax, whereas the second digit (minor version) indicates the highest minor version within that major version to which the sender is conformant (able to understand for future communication).[2] While HTTP's core semantics don't change between protocol versions, their expression "on the wire" can change, and so the HTTP version number changes when incompatible changes are made to the wire format.[2] Additionally, HTTP allows incremental, backwards-compatible changes to be made to the protocol without changing its version with defined extension points.

The protocol version indicates the sender's conformance with the set of requirements laid out in that version's specification. Each major version of HTTP also has its own syntax for communicating messages. A message consists of a few things: control data, a header lookup table, a potentially unbounded stream of content and the trailer lookup

table. The control data is used to describe and route the message. The header lookup table contains the name/value pairs that are used to extend control data and convey additional information about sender/message/content/context. Lastly the trailer lookup table contains name/value pairs for communicating information obtained while sending the content.

When messages are sent the framing and control data are sent first, followed by the header section and if the message includes content, the content is sent after the headers. Potentially the message can contain the trailer section, which will be sent last. The messages are expected to be processed as a stream and the purpose of that stream and its continued processing is revealed while being read. This means that the control data describes what is needed to know immediately and the header fields describe what is needed to know before the content. Lastly the trailer provides optional metadata, that was unknown prior to sending the content.[2]

The messages are intended to be so that everything a recipient needs to know about the message can be determined by reading the message itself, without requiring any outside information from the sender or otherwise. The client still must retain knowledge of the request when parsing, interpreting or caching a corresponding response since for example, responses to the HEAD method look just like the beginning of a response to GET but cannot be parsed in the same manner.[2] The framing of a message indicates the beginning and end the message, such that each message can be distinguished from each other or noise on the same connection. Almost all modern implementations use explicit framing in the form of length-delimited sequences of message data.[2]

The start of messages, the control data describe its primary purpose, request message control data includes a request method, request target and protocol version. A response message control data includes a status code, optional reason phrase and protocol version. Every HTTP message has a protocol version and recipients use that information to determine limitations or potential for later communication with that sender. When a message is forwarded by an intermediary the protocol version is updated to reflect the version used by that intermediary.

A client should send a request version of equal to the highest version which it can and whose major version is no higher than the highest supported by the server if it is known.[2] A client must not send a version to which it doesn't conform to, but may send a lower request version if it is known that the server has implemented a version of HTTP incorrectly, but only after a normal request has been attempted and failed due to improper handling of higher request version by the server. A server on the other hand

should send a response version equal to the highest it can that has a major version less than or equal to the one received. As the same with the client a server must not send a version it doesn't conform to.

2.3 QUIC vs TCP

Initially released by Google in 2013 [14], QUIC has been discussed in many studies since its conception [14] [15] [16] [18] and it can be considered the next step in transport-layer performance. This performance has had many improvements and studies on it, but these have focused on TCP [19] [19] [21] QUIC was then developed to replace TCP with a few goals, "QUIC is designed to meet several goals [22], including deployability, security, and reduction in handshake and head-of-line blocking delays."[23] As the implementation of QUIC in HTTP/3 is quite different, this section covers some essential differences and examples of how it affects HTTP communication. A good summary can be found on many sites, one example is the chapter "QUIC vs. TCP – Development and monitoring guide" [24] which also goes over many important points. Another good example with many visual examples is a blog post "The Evolution of HTTP: Refining Head-of-Line Blocking Solutions with HTTP/3" [25].

TCP is used for HTTP/2 and before since it is simpler and works well as the handshakes make sure the content delivered reliably. Though this reliability comes with downsides as this does create inefficiencies, notably speed and latency. For smaller payloads, which is generally most of web browsing, this might not be noticeable, but for performance orientated applications the impact is significant.

QUIC is designed to address these performance issues, it also provides a more adaptable system for communications. Unlike TCP QUIC uses the UDP, which means instead of handshakes happening all the time it is more stream orientated, this minimizes overheads for efficiency.

Differences in implementation of QUIC and TCP are that QUIC enables a 0-round trip time for establishing connection, while TCP requires a 3-way handshake to establish connection. QUIC by design supports multiple independent streams within a single connection, this reduces head-of-line blocking, while TCP handles streams sequentially. QUIC numbers all transmitted packets explicitly so that acknowledgements specify which packets have been received. TCP's acknowledgements implementation on the other hand is more ambiguous and leaves room for unclear acknowledgements, this can sometimes mean it is not certain if an acknowledgement was for new or retransmitted data. QUIC can also distinguish between actual congestion and transient network delay

to prevent unnecessary slowdowns. TCP's slower start method can lead to premature reductions due to misinterpreted congestion signals. There are papers looking into the security aspects of the 0-round trip time of QUIC and the TLS implementation [25] [27] [28] In general, QUIC outperforms TCP, but does have a few scenarios where this is not true. QUIC is sensitive to out of order packet delivery and can interpret such behaviour as loss, in which case it performs much worse than TCP. Another scenario is with resource constrained devices, where TCP can handle packet processing more efficiently than QUIC.

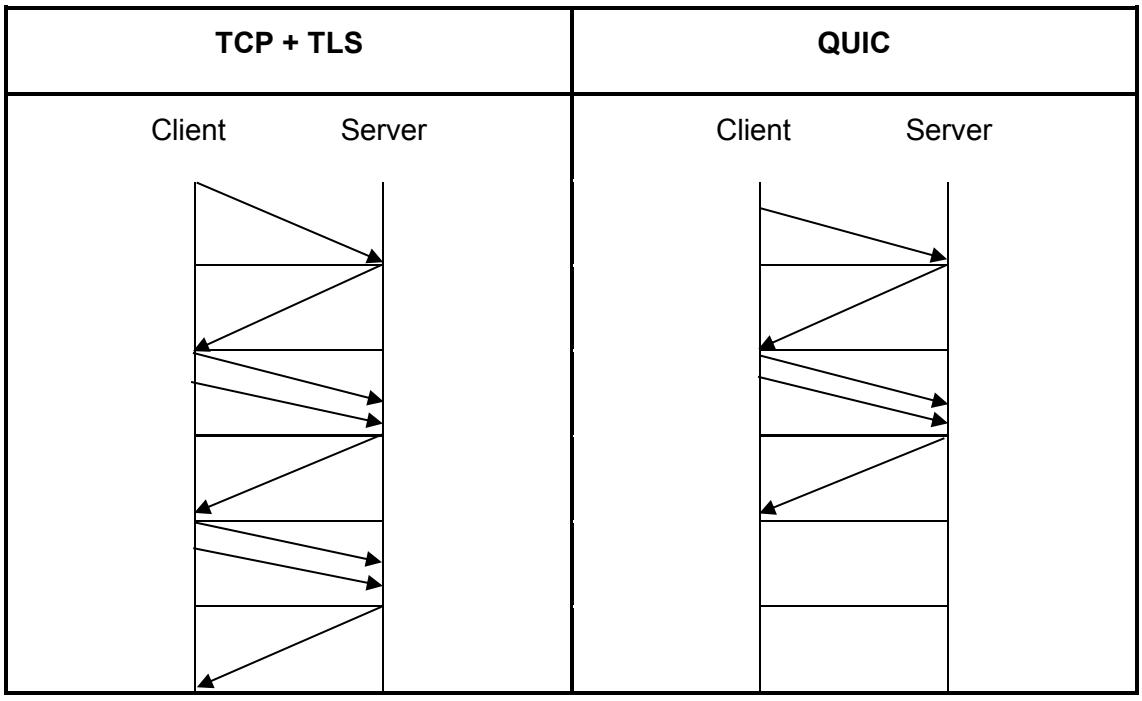

Table 1: Connection establishment and message exchange example TCP vs QUIC.

A visual example of a connection and message exchange is given in table 1. While the more intricate details are left out, it still shows how the differences between TCP and QUIC can play out and how QUIC is quite a bit faster and requires less work. The arrows represent packets sent and on the TCP plus TLS side, first is the 3-way handshake and right after this is the TLS negotiation. After that is the last two packets sent, which are the message exchange, which is the actual thing the client requested. On the QUIC side the connection is established first with just 3 packets and afterwards the two last packets are the HTTP request of the client and the response from the server. Notably this example doesn't even include the best scenario for QUIC, which would be the 0-round trip time scenario for connection establishment.

Another important feature of QUIC is how it deals with the Head-of-Line blocking that can affect HTTP/2.0. As many websites are quite large and host a varied amount of content, not all of it can fit in a single packet, meaning a client will send multiple GET packets to the server in order to load the whole website. Even in HTTP/2 this can be multiplexed quite effectively, meaning the client can send multiple requests one after the other before waiting for an answer from the server. Also the server can send an answer to the client requests in any order, if for example the last request was short enough that the server completes that request before any other it can send it back first without waiting on other requests to be completed. The problem of Head-of-Line blocking lies in the TCP side of HTTP/2 as it assembles the multiple responses from the server on the client side to be in order. If one of the packets were to be lost in transit TCP will wait for all the requests to be answered before combining them, so on the HTTP side this will simply be seen as a delay for the whole request. A visual example of a packet loss is shown below in table 2, with 3 requests being sent and the lost packet is represented as a dotted arrow and the retransmit request from the server is represented by the upward left facing arrow.

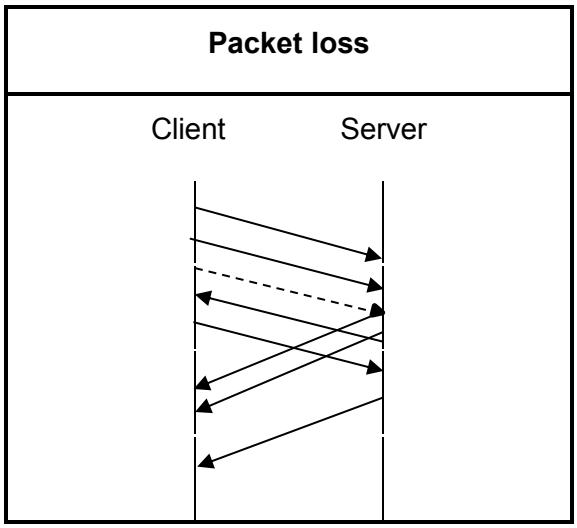

Table 2: Packet loss in a multiplexed request from a client.

So, in the event that a packet loss happens TCP will wait for the last response to arrive before it can combine them and process them further, this means nothing will load even if the packets themselves would be otherwise fully functional. QUIC however would be able to process each individual request and response as their own streams and would not need to wait for all of the packets to arrive. From the client's viewpoint using QUIC packet loss and retransmission would seem seamless and parts of the website could load independently while the retransmission would happen in the background and load

the contents of the lost packet last in the website. Multiple studies have also come to the same conclusion that in poorer networking environments QUIC fairs better [29] [30] [31] and in reliable and stable conditions the difference in results might not be so obvious.

3. HTTP HEADER FIELDS

As the focus of this study is on the header fields, we will first take a deeper dive into header fields themselves and then look at examples of the types of header fields. The "header section" of a message consists of a sequence of header field lines. Each header field might modify or extend message semantics, describe the sender, define the content, or provide additional context. [2] Fields that are sent or received before the content are referred to as "header fields" (or just "headers", colloquially).[2] HTTP uses these header fields to provide data in the form of extensible name/value pairs with a registered key namespace and these fields are sent and received within the header and trailer sections of messages.

The interpretation of a field does not change between minor versions of the same major HTTP version, though the default behaviour of a recipient in the absence of such a field can change. Unless specified otherwise, fields are defined for all versions of HTTP. New headers can be introduced without changing the HTTP version if its definition allows them to be safely ignored by recipients that don't recognize them.

Header fields can be repeated within the same section, as their names are case sensitive if the exact same field is presented more than once the combination for that field is the values chained together in order and separated by a comma. Thus, a recipient of a message with multiple header fields with the same name may combine them into one field without other changes to the message. This highlights the importance of the order in which the header fields are sent and received and that they must not be changed by proxies forwarding the message. Also, notably this means that when sending a message, the sender must not generate header fields with the same name unless the definition of the field allows it to have multiple values appended after another.

In terms of limits there is no set limit on the length of either the header field, its value or the header section itself. A server that receives larger than wanted header fields must respond with a client error message status code, as ignoring it could lead to increased server vulnerability. A client on the other hand may discard or truncate larger header fields if the dropped values can be safely ignored without changing the message framing or response.

As for the values in header fields, they are usually constrained to use US-ASCII (American Standard Code for Information Interchange) characters. Fields that use only a single member as a value can be referred to as "singleton fields". For header fields that allow

multiple members in their field value are referred as "list-based fields". The multiple members in a field value are separated by commas (",") and as some header field values allow commas being a part of the member, they need to be treated with care to detect possible errors and improve interoperability.

3.1 In depth look at header fields

HTTP headers are a pillar of HTTP messages. Servers and clients alike rely on the information exchanged in HTTP headers regardless of whether they are in the HTTP request or HTTP response. There are specific headers for both requests and responses alike as well as universal ones for either one. HTTP request-specific headers might contain information about the client and HTTP response-specific headers may contain details about the host. A representational HTTP header, for example, contains information about the body of the resource such as how it is encoded. There are also payload headers, which hold representation-independent information such as the size of the message body.

HTTP headers can be further characterized by their scope within the message chain. An end-to-end header that is sent by the client must be transmitted to the server, unmodified by intermediaries, and stored by caches as needed. On the other hand, a hop-by-hop header is only relevant between nodes, and must not be forwarded or stored by caches.

There are many different types of HTTP headers that describe or govern processes such as authentication, caching, message body information, and proxies. For example, Cookies are transmitted via HTTP headers, and they are responsible for maintaining the current client’s state in a stateless system.

3.2 Primary header field categories

Headers can be grouped according to their contexts, there are the 4 primary groups of headers.

Request headers that contain more information about the resource to be fetched, or about the client requesting the resource. This information is then used by the server to tailor its response to the request. An easy example of a request header would be the "Accept-" type headers that indicate which formats are allowed or preferred in the response.

Response headers hold additional information about the response, that doesn't directly relate to the content itself, like information about its location or about the server providing it. A good example is the "Location" header that provides the URL of a redirection page, which is used when a server responds with a redirection (code 3xx) or created (code 201) status response.

Representation headers contain information about the body of the resource, particularly the form of the resource. A representation is a form of a particular resource. An example is if a resource is localized to a particular written language, then a header called "Content-Language" is used to describe the language and allow for users to differentiate it according to their own preferred language. This can be done as the client sends a request and in it the client can use the header "Accept-Language" to specify the preferred languages, which are then sent back from the server with the representation headers telling the client the format of the selected representation they received.

Payload headers contain representation-independent information about payload data. This includes for example the header "Content-Length" which contains information about the length of the message payload.

Headers can also be grouped according to how proxies handle them, these are used to divide the HTTP headers in 2 categories for defining their behaviour with caching and non-caching proxies, these are end-to-end and hop-by-hop.

End-to-end headers must be transmitted to the final recipient of the message: the server for a request, or the client for a response. Intermediate proxies must retransmit these headers unmodified and caches must store them appropriately.

Hop-by-hop headers are used for only a single transport-level connection and must not be retransmitted by proxies or cached. Only hop-by-hop headers may be set using the "Connection" header.

3.3 Normal categories of header fields

This section will describe and list headers by their use cases and give a few examples for each category. Included are also a couple of headers that are either experimental or deprecated as examples of such cases. Experimental headers are ones that are not yet fully implemented and thus are not intended to be used in a normal use case. The following categories and examples are a current peek into what headers there are now as headers are ever evolving with new ones being made and older ones deprecating.

Places that have gathered lists of headers that include all or nearly all headers can be found on sites such as [32] and [33] for example.

Authentication headers is the first category, with headers like "WWW-Authenticate", a response header that is used to inform the client which authentication messages can be used to access a resource. Another header is "Authorization", a request header used to provide credentials to authenticate a user with a server, thus allowing access to a protected resource. Also included are headers like "Proxy-Authenticate", a response header that defines the authentication method that should be used to access a resource behind a proxy server and "Proxy-Authorization", a request header containing the credentials used to authenticate a user with a proxy server.

Caching is the next category, this includes headers like "Age", The response header contains the time in seconds, that the object has been in a proxy cache. "Cache-Control" a both request and response header that contains instructions that control caching in browsers and shared caches. "Clear-Site-Data" a response header that clears browsing data associated with the requesting website. Caching also includes the "Expires" response header that is used to tell the date/time after which the response is considered stale. Within the caching category there is also an experimental header the "No-Vary-Search Experimental", this experimental response header could be used to specify a set of rules that define how a URL's query parameters will affect cache matching. These rules would dictate whether the same URL with different URL parameters should be saved as separate browser cache entries. Another interesting one is the deprecated header "HTTP Pragma", this deprecated request/response header has different effects that are implementation-specific and is not included in the HTTP specification. However, it can be used for backward compatibility with HTTP/1.0 caches that do not have a HTTP Cache-Control HTTP/1.1 header.

The "Conditionals" category of headers include for example the "Last-Modified", The representation header used to contain the last modification date of the resource, used to compare several versions of the same resource. It is less accurate than "ETag", but easier to calculate in some environments. Conditional requests using If-Modified-Since and If-Unmodified-Since use this value to change the behaviour of the request. The "ETag" header also known as "entity tag" is a response header that contains an identifier for a specific version of a resource. Conditional requests using If-Match and If-None-Match use this value to change the behaviour of the request. The "If-Match" is a request header makes the request conditional, and the server will only return the requested resources if the stored resources match one of the given ETags. The "If-None-Match" is another request header that makes the request conditional and applies the method only

if the stored resource doesn't match any of the given ETags. This is used for example to update caches or to prevent uploading a new resource when one already exists. Also included in the conditionals category are the "If-Modified-Since" and "If-Unmodified-Since" request headers. These make the request conditional in that the server will only send the requested resource or accept it in the case that it either has been or hasn't been modified after the given date in the header.

Connection management is a category used for headers such as the "Connection" and "Keep-Alive" headers, which aren't allowed in HTTP/2 and HTTP/3 and are ignored. The "Connection" header is a request/response header used to control whether the network connection stays open after the current transaction finishes. And the "Keep-Alive" is a request/response header used to control how long a persistent connection should stay open.

Next is the "Content negotiation" category, with such headers as "Accept" a request header that informs the server about the types of data that can be sent back. There are also similar headers like "Accept-Encoding" a request header that is used to indicate the encoding algorithm, usually a compression algorithm, that can be used on the resource that is sent back. Another header is the "Accept-Language", this request header informs the server about the human language the server is expected to send back. This is a hint and is not necessarily under the full control of the user, the server should always pay attention not to override an explicit user choice (like selecting a language from a dropdown). An interesting one also included in the category is the "HTTP A-IM" request header, which is similar to HTTP Accept, except that it puts restrictions on the instance manipulations that are allowed in the HTTP response.

Controls is also a category of headers with examples like "Expect" a request header used to indicate expectations that need to be fulfilled by the server to properly handle the request. Another example is the header "Max-Forwards" that is needed when using the TRACE method, this request header indicates the maximum number of hops the request can do before being reflected to the sender.

There is a category "Cookies" for headers like "Cookie" and "Set-Cookie". "Cookie" is a request header that contains stored HTTP cookies previously sent by the server with the "Set-Cookie" header. The "Set-Cookie" is a response header that is used to send cookies from the server to the user.

A significant category of headers is the "CORS" or "Cross-Origin Resource Sharing" category. The Cross-Origin Resource Sharing is an HTTP-header based mechanism that allows a server to indicate any origins other than its own from which a browser should

permit loading resources. Examples include headers like "Access-Control-Allow-Credentials" a response header that indicates whether the server allows cross-origin HTTP request to include credentials. Another example is the "Access-Control-Allow-Headers" this response header is used with a pre-flight request (a CORS pre-flight request checks if the CORS protocol is understood.) to indicate which headers can be used during the actual request. "Access-Control-Allow-Origin" is a response header indicating whether the response can be shared with the requesting code from the given origin. The "Access-Control-Request-Headers" is the request header used when issuing a pre-flight request to let the server know which HTTP headers will be used when the actual request is made. The final example is the "Origin" request header that indicates the origin that caused the request.

The "Downloads" is a small category with an example header of "Content-Disposition". In a normal response this response header indicates if the content is expected to be displayed as a part or of a webpage or as an attachment. In a subpart of a multipart body this response/request header must be used to give information about the field it applies to.

Message body information is another category with headers like "Content-Length" the request/response/payload header that indicates the size of the resource in bytes. Similar headers included in this category are the "Content-(Type/Encoding/Location)" These representation headers are used to indicate: the media type of the resource prior to encoding (Type), the lists of any encodings that have been applied (Encoding) and the alternate location for the returned data (Location). Also included is the "Content-Language" representation header that describes the human language(s) intended for the audience, so that it allows a user to differentiate according to the users' own preferred language.

The "Proxies" category includes headers such as "Forwarded" a request header used for debugging that contains information from the client-facing side of proxy servers that is otherwise altered or lost when a proxy is involved in the path of the request. "Via" is another header and it is a request/response header added by proxies, it is used for tracking message forwards, avoiding request loops and identifying protocol capabilities of senders along the chain.

The "Redirects" category is another small one containing the header "Location". The "Location" response header is used to indicate the URL to redirect a page to.

Headers belonging to the "Request context" category include the "Host" request header that specifies the domain name of the server and optionally the TCP port number on

which the server is listening. "Referer" The request header that contains the address from which a resource has been requested. The "Referrer-Policy" is a Response header that governs how much referrer information sent in the "Referer" header should be included with requests made. Lastly mentioned in this category is the header "User-Agent", this request header contains a characteristic string that allows the network servers and peers to identify the application type, operating system, software vendor or software version of the requesting user agent.

"Response context" on the other hand is a category with headers like "Allow", the response header that lists the set of HTTP request methods supported by a resource. Another is the "Server" response header that contains information about the software used by the origin server to handle the request.

A category dealing with ranges is the "Range requests" category with headers like "Accept-Ranges" a response header that indicates if the server supports range requests, and if so in which unit the range can be expressed. "Range" request header indicating the part of a resource that the server should return. "If-Range" a conditional request header that if fulfilled the server sends pack an answer with the appropriate body, if not fulfilled the full resource is sent back. Lastly the header "Content-Range" a response/payload header that indicates where in a full body message a partial message belongs.

The "Security" category deals with headers such as "Cross-Origin-Embedder-Policy (COEP)", a response header that allows a server to configure embedding cross-origin resources into a document. "Cross-Origin-Opener-Policy (COOP)", the response header that isolates a document and prevents other domains from opening/controlling a window. "Content-Security-Policy (CSP)", this response header controls resources the user agent is allowed to load for a given page. The "Permissions-Policy" response header is responsible for providing a mechanism to allow and deny the use of browser features in a website's own frame, or in embedded frames. Also included are the headers "Strict-Transport-Security (HSTS)", a response header used to force communication using Hypertext Transfer Protocol Secure (HTTPS) instead of HTTP. Lastly another notable one is "Upgrade-Insecure-Requests" the request header that sends a signal to the server expressing the client's preference for an encrypted and authenticated response, and that it can successfully handle the upgrade-insecure-requests directive.

The "Fetch metadata request headers" is another unique category, as they provide information about the context from which the request originated. A server can use them to make decisions about whether a request should be allowed, based on where the request

came from and how the resource will be used. Examples from this category are the headers "Sec-Fetch-Site" that indicates the relationship between a request initiator's origin and its target's origin. Basically, this header tells whether a request for a resource is coming from the same origin, the same site, a different site or is a "user initiated" request. The header "Sec-Fetch-User" is used only when requests are initiated by user activation, a server uses this to identify whether a navigation request from a document originates from the user. The "Sec-Fetch-Dest" indicates the request's destination, which is the initiator of the original fetch request. The server uses this to determine whether to service a request based on how appropriate it is for the given destination. For example, a request from an audio destination should request audio data and not any other type. Lastly the header "Sec-Purpose", this header indicates the purpose of the request and how the resource will be used if it's something else than immediate use by the user.

The last specific category is the "Transfer coding" category, that includes headers such as "Transfer-Encoding" a very general header. It can be considered a request/response/payload header and is also a hop-by-hop header as it specifies the form of encoding used to safely transfer the resource payload body to the user and is applied to a message between two nodes, not to a resource itself. "TE" is another one, this request header is used to specify the transfer encodings the user is willing to accept. In HTTP/2 and HTTP/3 it is only used if the trailers value is set. Lastly the "Trailer" request/response/payload header is used to allow the sender to include additional fields at the end of chunked messages, used to supply dynamic metadata that is generated while the message body is sent.

In addition, there is the "other" category, the headers belonging to the "other" category are unique as themselves and thus don't belong to another category as such. These headers include the "Alt-Svc" header that allows a server to indicate another network location that can be treated as authoritative for that origin when making future requests. The "Alt-Used" header is used in requests to identify an alternative service in use, much like the "Host" header, it contains the name and the optional port of the host. This is used to allow alternative services for purposes such as load balancing. "Date" is a simple request/response header that contains the date and time at which the message originated. "Retry-After" is a response header that indicates how long the user should wait before making a follow-up request. "SourceMap" is a response header that links generated code to a source map, which enables the browser to reconstruct the original source and present the reconstructed original in the debugger. Last example is the "Upgrade" request/response header that is used exclusively in HTTP/1.1 and is used to either upgrade the connection from HTTP/1.1 to HTTP/2.0 or an HTTP or HTTPS connection into

a WebSocket (a two-way interactive communication session between the user's browser and a server).

3.4 Additional categories of header fields

In addition to the list of many headers that are generally used mentioned above, a brief explanation of other unusual categories not used in general. This will be a more brief look into them as they have more specific use cases outside the norm.

First are the experimental headers and as the name implies, they are not fully implemented yet and some perhaps never will be or are not meant to be implemented normally. In addition to previous normal categories that can have experimental headers, there are a few categories considered as experimental themselves and only contain experimental headers. These experimental categories include "Attribution reporting headers", "Client Hints", "User agent client hints", "Device client hints", "Network client hints", "Privacy" and other individual headers not belonging to any category thus labelled as "Other"

Attributions reporting headers are used by developers to measure conversions, like when a user clicks an ad embedded on a site. It does this without relying on third-party tracking cookies and just by relying on various headers to register sources and triggers matched to indicate a conversion.

Client hints are a set of request headers which provide useful information about the client such as device type and network conditions, these can be used by the server to optimize serving in those conditions. User agent client hints are request headers that provide information about the user agent such as platform/architecture and user preferences. These can then be used by the server to vary responses based on the hints. Device client hints are much the same but related to the device itself such as "Device-Memory" header that is used to approximate the amount of available memory.

Network client hints are headers that allow a server to choose what information is sent based on the user's choice and network bandwidth and latency combined. Privacy headers are used to restrict information access from third parties according to the user's consent.

There are also some headers, which are considered as "Non-standard headers". One example is the "X-Forwarded-Proto" a request header that identifies the protocol that a client used to connect to the proxy.

3.5 Captured HTTP packets

In this last part for HTTP header fields, HTTP packets have been captured by using the software Wireshark[34] to show examples of how header fields are used in practice. Examples are shown for each major version of HTTP (1.1/2/3) to show the differences between the versions as well.

Figure 1: HTTP/1.1 Packet sent from client to server

In the figure above is shown a request sent from a client to a server and as is shown under the highlighted GET method, the version of the request is HTTP/1.1. After the GET method's own data there are the HTTP header fields starting with "Host" and ending with "Accept-Language".

Figure 2: HTTP/1.1 Packet from the server to the client

In figure 2 we have the answer from the server to the previous request as seen in figure 1. Status code is 200 so the request was successfully answered and given the response OK. Other than that, in these quite simple packets are the headers starting from "Access-Control-Allow-Origin" and ending at "Content-Type". Though notably the headers "Date" and "Expires" are interesting as the date of the request I made is from 2024 and the expiry is in 1990 as well as the "Last-Modified" being in 1998. While this HTTP/1.1 packet is inconsequential as it is an automated packet, probably sent alongside the actual site accessed as deduced by the "Request URI" having "google-analytics" in it. Still intriguing as the dates seem to be from even before the founding of Google itself.

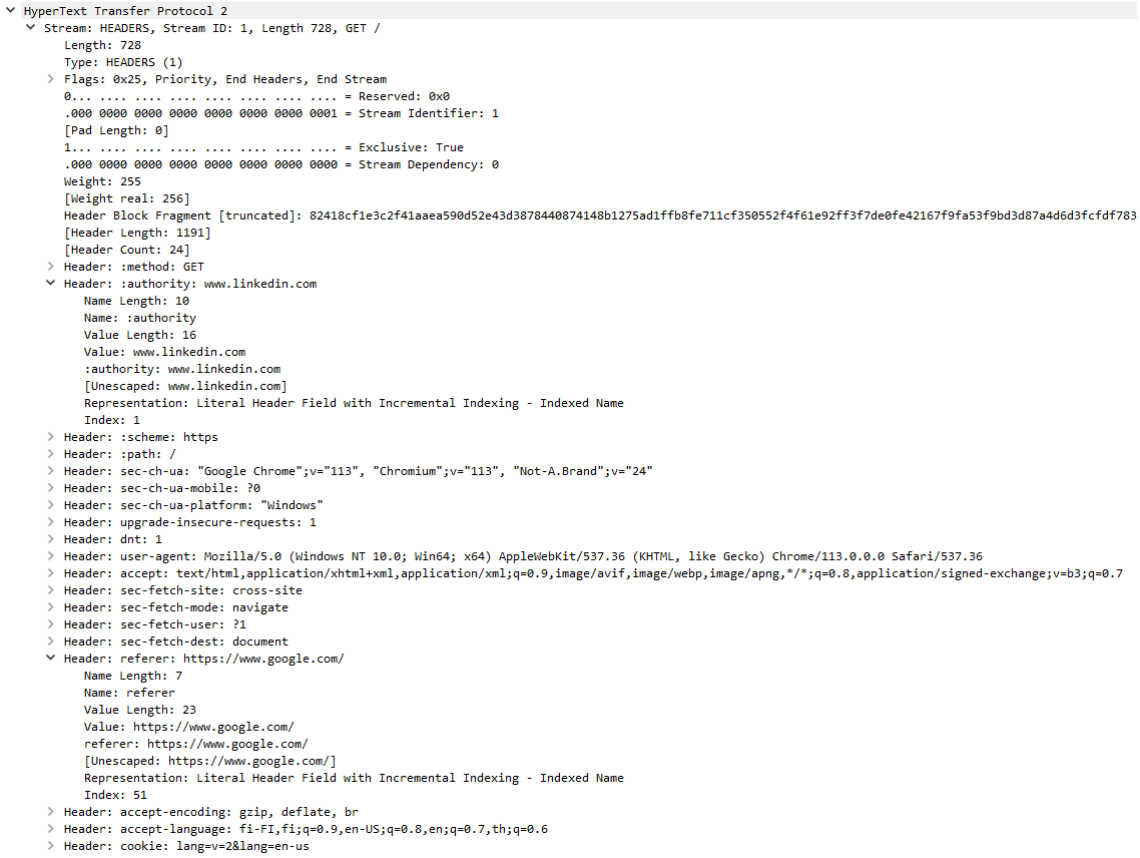

Figure 3: HTTP/2 packet from client to server

The first difference in packets when comparing HTTP/1.1 and HTTP/2 is the how the packets are sent, because in HTTP/2 HPACK and encoding are used, meaning that the packets themselves are no longer simply visible. To see these packets, one must know the key used to be able to decrypt it and see the packet as what is shown above in figure 3. The packet in figure 3 is once again a request from the client. The other notable difference is before the headers themselves is a new "Header" tag, noting that what comes after is indeed the header itself. Also in HTTP/2 a difference is that each header now has a list of its attributes for example as in figure 3 the first opened header "authority" has the attributes "Name Length" being 10 and then the name itself followed by its value's length and the value itself. The last attributes for the "authority" header are the representation of the header and its index. This time as the request was made via another site, a "referer" header is present. The "referer" is Google this time as seen by the value of the header and the request was indeed made after searching the site on Google and clicking its link.

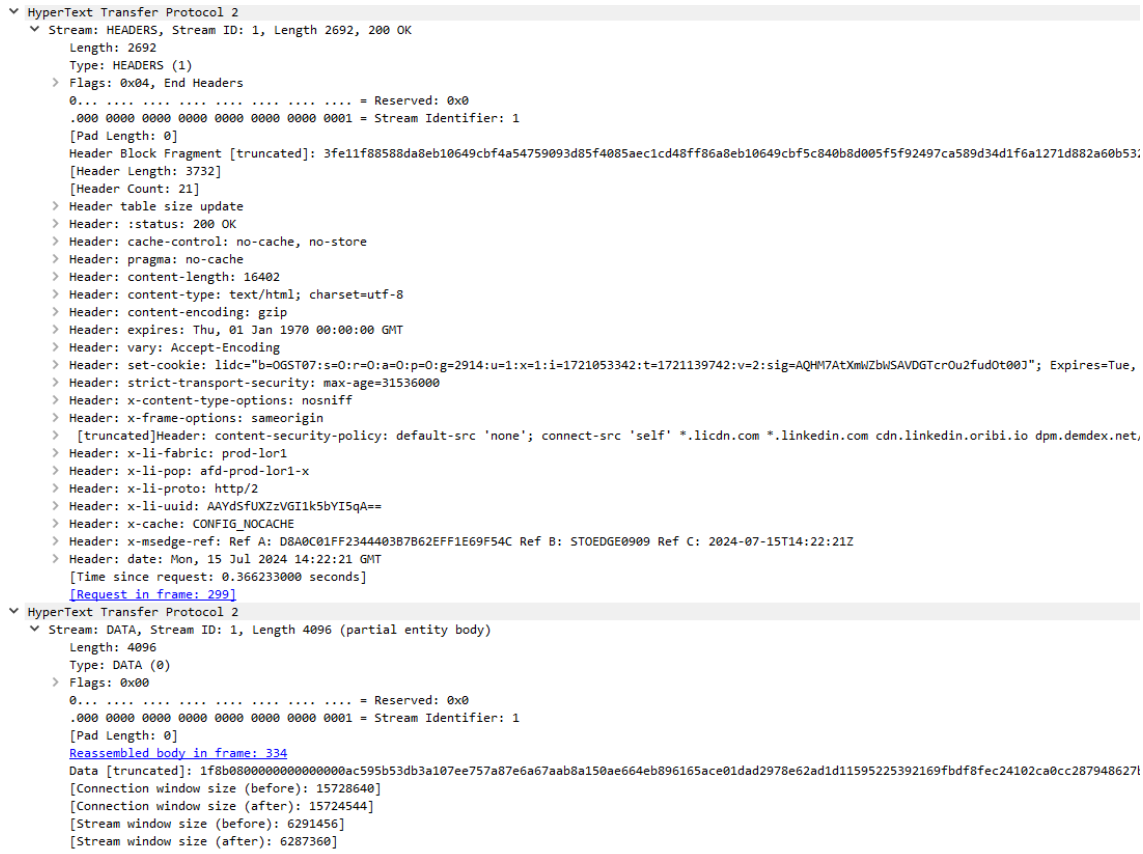

Figure 4: HTTP/2 packet sent from server to client

Figure 4 shows the response from the server to the previous request in figure 3. Right before the header fields a line with the information "Header table size update" which refers to the dynamic table of headers being updated and communicated to the client. One notable header is the encoding, as previously in figure 3 the request sent that it would accept a few forms of encoding (gzip, deflate, br) and the server has chosen the "gzip" method of encoding the data as seen with the header "content-encoding: gzip". Another interesting note is in the header "content-security-policy" which has a [truncated] prepended on it meaning the header values were larger than the default of 240 characters in Wireshark. This does not mean the header value went over a size limit of the header itself as that is much larger. Perhaps the most interesting part of this response is the second part after the headers, being the stream of data. This data stream is the body of the response holding the requested information. The reassembled body mentioned in this data stream is shown in figure 5 below.

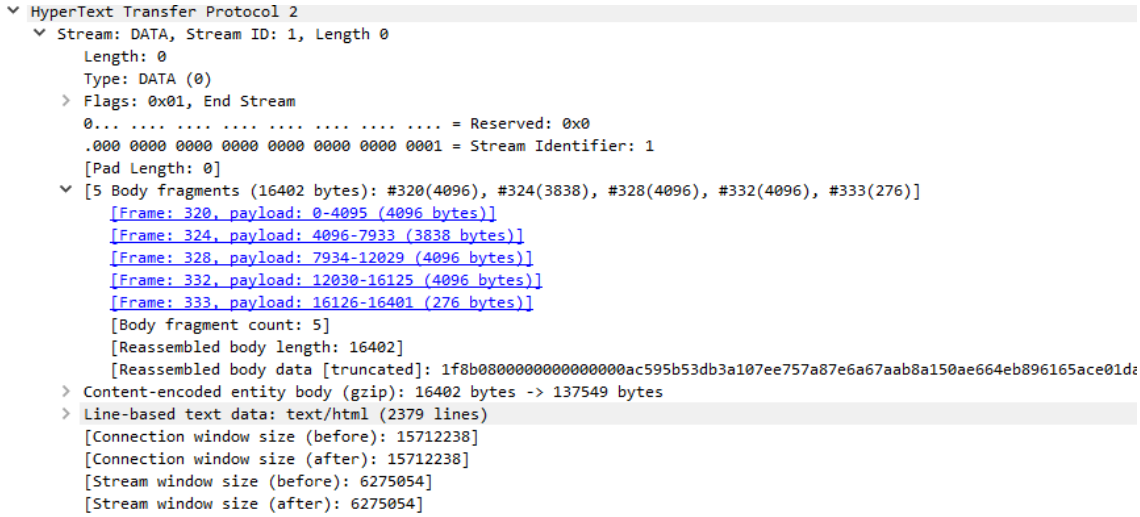

Figure 5: HTTP/2 packet showing payload data

Figure 5 shows the payload that the serve sent in figure 4. This is the reassembled body from the multiple payloads sent by the server, with a total count of 5 payloads. As each payload has a size limit of 4KB (Kilo Bytes) the payloads were split to be either at or under this limit. Notably the information after the reassembly shows the total length being 16402 bytes, but this is with the encoding as a bit below this it shows the "Content-encoded entity body (gzip)" meaning it was the gzip encoded size with the actual payload having a size of 137549 bytes. This is a total reduction of around 88%, which is quite impressive and shows how efficient compression can be.

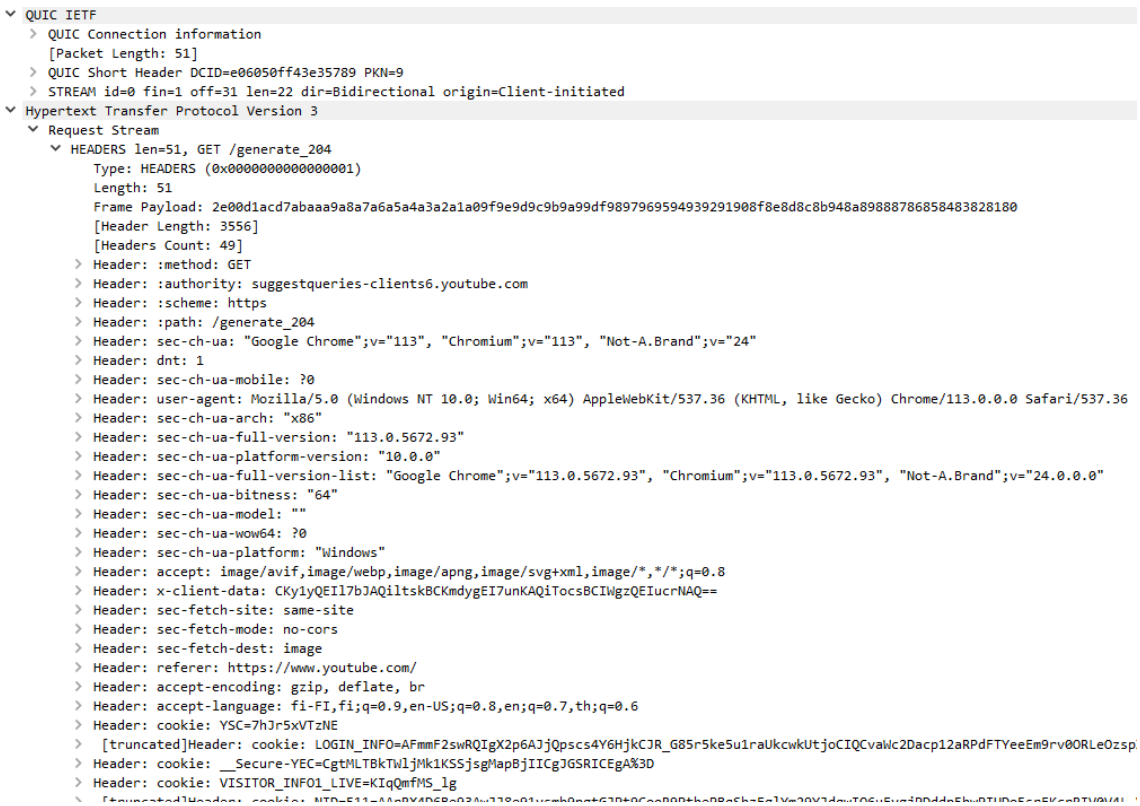

Figure 6: HTTP/3 packet sent from client to server

In figure 6 we finally have a packet from a request made in HTTP/3. And same as with HTTP/2 it was encrypted and needed the key used to be able to decrypt it and show its information as seen in figure 6. The first difference to previous ones is the QUIC protocol used in HTTP/3 to allow multiple streams of data to occur with this request starting a bidirectional stream with the server. Each stream having their own identifier the first one is "id=0" as seen in the figure above. Below it is the HTTP/3 packet itself with all the headers, totaling to 49 different headers.

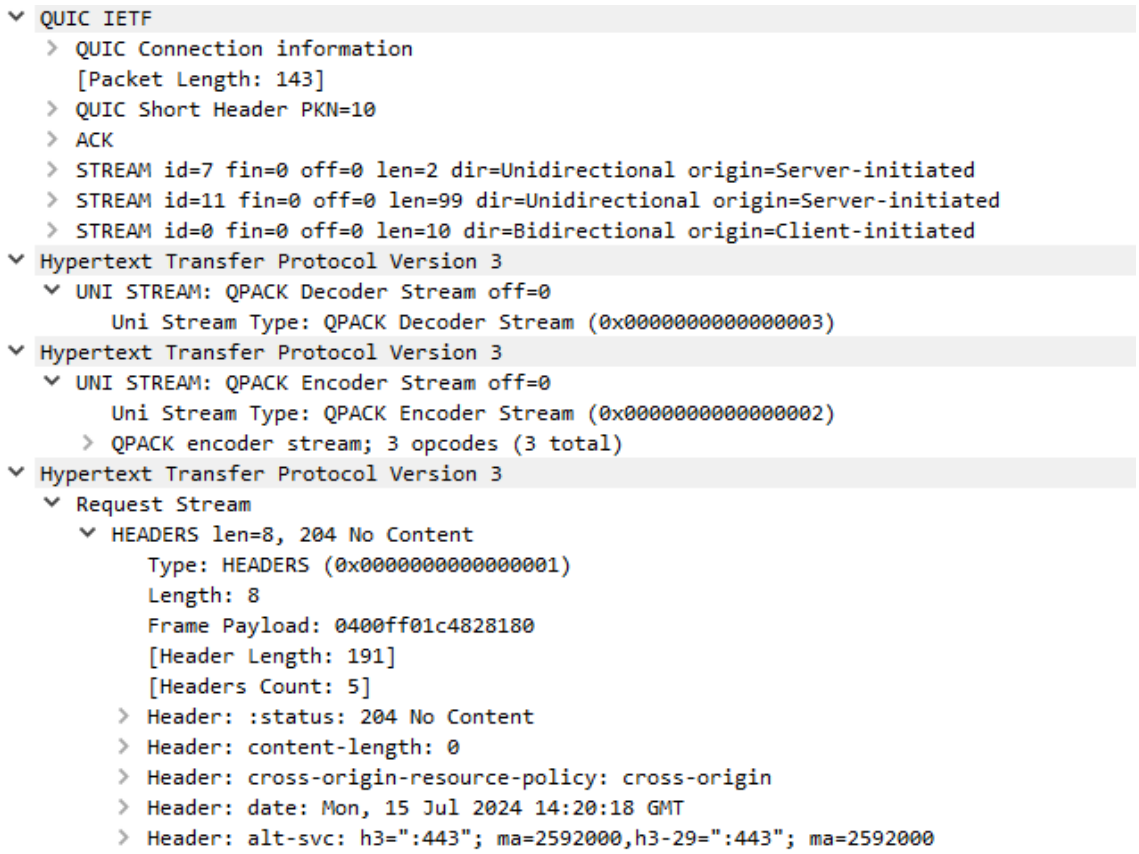

Figure 7: HTTP/3 packet sent from server to client

In figure 7 we see a response packet to the request seen in figure 6. First in the QUIC side is an ACK of the request and then the server starting 3 different streams, one of which is the continuation of the first bidirectional stream with the "id=0" which is also noted with the "origin=Client-initiated" as opposed to the two new streams with "origin-Server-initiated" on them. Notably the other two streams are unidirectional and having non sequential id's the streams being ones with "id=7" and "id=11". Below the QUIC part are the HTTP/3 streams with the last one being part of the request and the only one given header fields as a response.

4. COMPRESSION ALGORITHMS AND PERFORMANCE

This chapter will discuss and compare the compression algorithms used in HTTP, the first being HPACK introduced in HTTP/2 and then QPACK introduced in HTTP/3. Going over the features of the compression algorithms according to their documentations and security considerations in implementation. Lastly are performance evaluations of different HTTP versions and compression algorithm implementations.

4.1 Compression algorithms

Compression algorithms are used to cut down on data usage whether it is in storage or bandwidth. For the application of HTTP header fields, the compression algorithms are used to reduce the number of bits needed to send while still retaining all the necessary information required to perform the wanted operations.

Before going into the algorithms, here is some previously unmentioned terminology that applies to both HPACK and QPACK and these help to understand the more intricate parts of the compression algorithms.

Static Table: is a table in which there are static pairs of header field names and values which occur frequently, this is always accessible and can be used by all encoding and decoding context.

Dynamic Table: is a table that contains dynamically added pairs of header field names and values that are used in specific encoding or decoding context.

Header Field Representation: A header field in encoded form either as literal or as indexed in a table.

Header Block: An ordered list of header field representations, when decoded, yields a complete header list.

Header List: a collection of ordered header fields that are encoded jointly.

4.2 HPACK

Since in HTTP/1.1 header fields were not compressed and thus consumed a lot of bandwidth unnecessarily, HPACK was introduced in HTTP/2 to solve this. HPACK was defined as a new compressor that would eliminate redundant header fields, limit vulnerability to known security attacks and had bounded memory requirement for use in constrained environments [35]. HPACK was made to be simple and inflexible to reduce risks of interoperability or security issues due to wrong implementations. In addition, this means there are no defined ways to offer extensibility and the only way to make changes to the format is defining a complete replacement. The following parts mostly follow the RFC documentation for HPACK [35], but does not include every detail, so it can be viewed for more in depth information.

In the RFC 7541 [35], the format for the HPACK defines a list of header fields as an ordered collection of name-value pairs that can include duplicate pairs. Names and values are considered to be opaque sequences of octets, and the order of header fields is preserved after being compressed and decompressed [35]. Header field tables are updated as new header fields are encoded or decoded. In the encoded form, a header field is represented either literally or as a reference to a header field in one of the header field tables [35]. So, a list of header fields can be encoded using either references or literal values or both. Literal values are either encoded directly or use a static Huffman code. The encoder decides which header fields to insert as new entries in the header field tables. The decoder on the other hand executes the modifications to the header field tables prescribed by the encoder and reconstructs the list of header fields in the process.

HPACK preserves the ordering of header fields inside the header list. This means that an encoder has to order header field representations in the header block in the same order as they appeared in the original header list. A decoder then must also order header fields in the decoded header list according to the order they appear in the header block. To decompress header blocks, a decoder only needs to maintain a dynamic table as a decoding context. When used for bidirectional communication, such as in HTTP, the encoding and decoding dynamic tables maintained by either side are independent. This means the request and response dynamic tables are separate from each other.

HPACK uses two tables for associating header fields to indexes. The static table is predefined and contains common header fields (most of them with an empty value). The dynamic table is used by the encoder to index header fields repeated in the encoded header lists and is updated during communication. In communication these two tables are combined into a single address space for defining index values.

The dynamic table list of header fields is maintained in a first-in, first-out order. The first and newest entry in a dynamic table is at the lowest index, and the oldest entry is at the highest index, a visualization of this is in the table 3 below. The dynamic table is initialized as empty and entries are added as each header block is decompressed. The dynamic table can contain duplicate entries, that have the same name and same value. Therefore, duplicate entries must not be treated as errors by a decoder. The encoder decides how to update the dynamic table and thus controls how much memory is used by the dynamic table. The dynamic table size is strictly bounded to limit the memory requirements of the decoder. Reduction in the maximum size of a dynamic table can cause entries to be evicted.

Index Address Space					
Static Table			Dynamic Table		
1	…	s	s+1	…	s+k
			^ Insertion point		^ Drop point

Table 3: Index Address Space visual diagram [35]

A decoder processes a header block sequentially to reconstruct the original header list, the header block is the ordered list of header field representations. Once a header field is decoded and added to the reconstructed header list it cannot be removed. Once a header field is added to the header list it can be passed to the application. By passing the resulting header fields to the application, a decoder can be implemented with less memory commitment in addition to the memory needed by the dynamic table. Protocols that use HPACK choose the maximum size that the encoder can use for the dynamic table. There are two primitive types that HPACK encoding uses: unsigned variable-length integers and strings of octets.

Integers are used to represent either name indexes, header field indexes or string lengths. An integer representation can start anywhere within an 8-bit sequence called an octet. An integer representation always finishes at the end of an octet as this allows for more optimized processing. There are two parts in an integer representation: a prefix

that fills the current octet and an optional list of octets that are used if the integer value does not fit within the prefix. The table below shows an example of an encoded integer value, where the prefix (N) in bits is 5. If the integer value is small enough, strictly less than 2N-1, it is encoded within the N-bit prefix, as is the case in this example shown below, the first row represents the index of bits in an octet (byte).

0	1	2	3	4	5	6	7
x	x	x	Value				

Table 4: Integer value encoded with a prefix of 5 (N=5) [35]

If the integer value is larger then all the bits of the prefix are set to 1 and the value, decreased by 2N-1, is encoded using a list of one or more octets. The most significant bit of each octet is used as a continuation flag: its value is set to 1 except for the last octet in the list, as can be seen in the first column of table 5. The remaining bits of the octets are used to encode the decreased value. An example for encoding a larger integer value is shown in the table below with a prefix of 5.

0	1	2	3	4	5	6	7
x	x	x	1	1	1	1	1
1	Value-(2^N-1) Least Significant Bit						
…							
0	Value-(2^N-1) Most Significant Bit						

Table 5: Integer value encoded after the prefix of 5 (N=5) [35]

Decoding the integer value from the list of octets starts in reverse order, from the last octet to the first. For each octet, its most significant bit is removed and then the remaining bits of the octets are concatenated. The resulting value is then increased by 2N-1 to obtain the integer value. The prefix size N is always between 1 and 8 bits and an integer starting at an octet boundary will have an 8-bit prefix.

This integer representation allows for values of indefinite size. It is also possible for an encoder to send a large number of zero values, which wastes octets and could also be used to overflow integer values. Integer encodings that exceed implementation limits in value or octet length must be treated as decoding errors.

The other way of representing header field names and header field values are as string literals. A string literal is encoded as a sequence of octets, either by directly encoding the string literal's octets or by using a Huffman encoding.

Huffman-encoded data doesn't always end at an octet boundary, so there is some padding inserted after it, up to the next octet boundary. To prevent this padding from being misinterpreted as part of the string literal, the most significant bits of the code corresponding to the EOS (end-of-string) symbol are used. Upon decoding, an incomplete code at the end of the encoded data is to be considered as padding and discarded. As an octet is only 8 bits, a padding longer than 7 bits must be treated as a decoding error. Another decoding error for padding would be when it does not correspond to the most significant bits of the code for the EOS symbol. Lastly a Huffman-encoded string literal containing the EOS symbol must also be treated as a decoding error.

Now for the binary format. An indexed header field representation identifies an entry in either the static or dynamic table and causes a header field to be added to the decoded header list. An indexed header field starts with a '1' 1-bit pattern followed by the index of the header field represented as an integer with a 7-bit prefix as seen below in the example table 6. Also notable is that the index value of 0 is not used and must be treated as a decoding error if found.

0	1	2	3	4	5	6	7
1	Index (7+)						

Table 6: Indexed header field [35]

As for the literal header field representation, it contains a literal header field value and the header field names are provided as literal or by reference to an existing table entry. There are 3 forms for this, with indexing, without indexing and never indexed.

Firstly, a literal header field with incremental indexing representation results in appending a header field to the decoded header list and inserting a new entry to the dynamic table.

A literal header field with incremental indexing representation starts with a '01' 2-bit pattern. If the header field name matches one in either the static or dynamic table, the header field name can be represented using the index (which is always non-zero) of the entry as an integer with a 6-bit prefix. Otherwise the header field name is represented as a string literal and a value of 0 is used in place of the index followed by the header field name. In both cases after either the index or header field name is the header field value represented as a string literal. An example of how a new header field with a new name is added is seen below in table 7. The name and value lengths mean the number of octets used to encode the literal, encoded as an integer with a 7-bit prefix, the 'H' before the names is an indicating bit as to whether the octets of the string are Huffman encoded or not. Lastly the strings themselves are the encoded data, if 'H' is 0 then the encoded data is the raw octets, if 'H' is 1 then the encoded data is the Huffman encoding of the string.

0	1	2	3	4	5	6	7
0	1	0					
H	Name length (7+)						
Name String (Length octets)							
H	Value length (7+)						
Value String (Length octets)							

Table 7: Literal header field with incremental indexing for a new name header field [35]

Next is the literal header field without indexing representation, this means appending a header field to the decoded header list without altering the dynamic table. Compared to before, this starts with a '0000' 4-bit pattern, otherwise following the same style as before. If the header field name matches an entry in either the static of dynamic tables it can be represented using the index of that entry, this time represented as an integer with a 4-bit prefix, being always non-zero just as before. Otherwise a value of 0 is again used for new header field names in place of the index and an example of a new name literal header field without indexing is shown below in table 8.

0	1	2	3	4	5	6	7
0	0	0	0	0			
H	Name length (7+)						
Name String (Length octets)							
H	Value length (7+)						
Value String (Length octets)							

Table 8: New name literal header field without indexing [35]

Lastly the literal header field never-indexed representation that just like without indexing results in appending a header field to the decoded header list without altering the dynamic table. If never-index is used, the intermediaries must also use the same representation for encoding the header field. This time the representation starts with a '0001' 4-bit pattern and when a header field is represented this way it must always be encoded with this specific literal representation. When a peer receives a literal header field never indexed it must use the same representation to forward the header field. This representation is used for protecting header field values that must not be put at risk by compressing them. The encoding representation is otherwise identical to a the one without indexing, just starting with the '0001' pattern instead of '0000' as seen in table 6.

Now for some potential areas of security concerning HPACK. One is using HPACK's compression for verifying guesses about secrets that are compressed into a shared compression context. Another is the denial of service from exhausting processing or memory capacity at a decoder. Also, HPACK reduces the length of header field encodings, so it can be exploited using the redundancy inherent in protocols like HTTP.

The compression context used to encode header fields can be probed by an attacker. The attacker can define header fields to be encoded and transmitted as well as observe the length of those fields once they are encoded. When an attacker can do both, they can adaptively modify requests and then make guesses about the dynamic table state. For example, if a guess is compressed into a shorter length, the attacker can observe the encoded length and deduce that the guess was correct. This is possible even over

the TLS protocol, because while TLS provides confidentiality protection for content, it doesn't really protect for the length of that content.

Padding schemes themselves only provide limited protection against an attacker with these capabilities. These could only force an increased number of guesses to learn the length associated with a given guess. Padding schemes also increase the amount of bits needed for transmission, thus working directly against compression.

Attacks like CRIME (Compression Ratio Info-leak Made Easy) demonstrated the existence of these general attacker capabilities. This permitted the attacker to confirm guesses a character at a time, reducing an exponential-time attack into a linear-time attack.

HPACK mitigates but does not completely prevent attacks modelled on CRIME by forcing a guess to match an entire header field value rather than any individual characters. This means attackers could only learn whether a guess was correct or not, reducing them to brute-force guesses for the header field values. For attackers the viability of recovering specific header field values would depend on the entropy of values. Values with high entropy would be unlikely to be recovered successfully and values with low entropy would remain vulnerable.

Attacks of this nature are possible any time that two mutually distrustful entities control requests or responses that are placed onto a single HTTP/2 connection. This means if the HPACK compressor permits one of the entities to add entries to the dynamic table and the other to access those entries, then the state of the table could be learned. Having requests or responses from mutually distrustful entities occurs when an intermediary either: sends requests from multiple clients on a single connection toward an origin server or takes responses from multiple origin servers and places them on a shared connection toward a client.

On the web browser side, they need to assume that requests made on the same connection by different web origins are made by mutually distrustful entities. Users of HTTP that require confidentiality for header fields could use higher entropy values to make guessing infeasible. However, this hinders communication for everyone because it forces all users of HTTP to take these steps to mitigate attacks and it would impose new constraints on how HTTP is used.

Rather than impose constraints on the users of HTTP, an implementation of HPACK could instead constrain how compression is applied to deter dynamic table probing. An ideal solution would segregate access to the dynamic table based on the entity that is constructing the header fields. Header field values that are added to the table would be

attributed to an entity, and only that entity could extract it. Improving compression performance for this option, could include tagging certain entries public. For example, a web browser might make the values of the "Accept-Encoding" header field available in all requests.

Another way of effectively preventing guesses would be that an encoder would introduce a penalty for a header field with many different values. This would mean that a large number of attempts to guess a header field value would result in the header field no longer being compared to the dynamic table entries, effectively preventing further guesses.

Simply removing entries corresponding to the header field from the dynamic table might not be most effective if the attacker has a reliable way of causing values to be reinstalled. For example, loading an image in a web browser typically includes the "Cookie" header field and web sites can easily force an image to be loaded, thus refreshing the dynamic table entry.

Implementations could also choose not to compress sensitive fields and instead encode their values as literals. Refusing to generate an indexed representation for a header field is only effective if compression is avoided on all hops. These are the never-indexed literals that can be used to signal to intermediaries that a value was intentionally sent as a literal. As mentioned before this means that the intermediary must not re-encode a value that uses this representation with another representation that would index it. If HPACK is used for re-encoding, then the never-indexed literal representation must be used.

Using never-indexed literal representations for header fields depend on a few factors. Since HPACK doesn't protect against guessing entire header field values, short or low-entropy values are easier to be recovered by an attacker. Therefore, an encoder could and should choose not to index values with low entropy. An encoder might also choose not to index values for are highly valuable or sensitive fields, such as the "Cookie" or "Authorization" header fields.

On the other hand, an encoder might prefer indexing values for header fields that have little or no value if they were exposed. For example, the "User-Agent" header field does not commonly vary between requests and is sent to any server and in that case, confirmation that a "User-Agent" value has been used provides little value. The criteria for deciding the use of a never-indexed literal representation can still evolve over time if new attacks are discovered.

There is currently no known attack against static Huffman encoding**.** A study has shown that using a static Huffman encoding table created an information leakage [36] however,

this same study concluded that an attacker could not take advantage of this information leakage to recover any meaningful amount of information.

An attacker could also try to cause an endpoint to exhaust its memory. HPACK is designed to limit both the peak and state amounts of memory allocated by an endpoint. The amount of memory used by the compressor is limited by the protocol using HPACK, this is done by defining the maximum size of the dynamic table. This limit considers both the size of the data stored in the dynamic table, as well as an additional small overhead. A decoder can set an appropriate maximum size for the dynamic table, to limit the amount of state memory used. An encoder on the other hand can limit the amount of state memory it uses by signalling a lower dynamic table size than the decoder allows.

As for temporary memory, the amount consumed by either the encoder or decoder can be limited by sequentially processing the header fields. An implementation does not need to retain a complete list of header fields. Even though HPACK doesn't force this, applications on the other hand might have constraints that would make retaining the complete list necessary.

4.3 QPACK

QPACK is a compression algorithm for compressing fields in HTTP/3 and it is a variation of HPACK that seeks to reduce head-of-line blocking. As HPACK is used in HTTP/2 for compressing header and trailer sections, if used in HTTP/3 it would cause head-of-line blocking for field sections because of the built-in assumptions of a total ordering across frames on all streams. As a variation of HPACK, QPACK uses the same core concepts from HPACK, but it is designed for flexibility in implementation and allowing correctness in out-of-order delivery. QPACK is also designed to strike a balance between resilience against head-of-line blocking and optimal compression ratio. This section contains information mostly from the RFC documentation for QPACK [37], but does not contain everything in the documentation, so it can be viewed for additional information.

The first major difference from HPACK is that QPACK defines two unidirectional stream types, an encoder stream is a unidirectional stream of type 0x02. 0x02 is simply an identifier for the stream type as many types are listed for use in HTTP/3. In this case it carries an unframed sequence of encoder instructions from encoder to decoder. A decoder stream is a unidirectional stream of type 0x03 and it carries an unframed sequence of decoder instructions from decoder to encoder.

HTTP/3 endpoints contain both a QPACK encoder and a QPACK decoder. Each endpoint must initiate at a maximum of one encoder stream and a maximum of one decoder

stream. Receipt of a second instance of either stream type is to be treated as a stream creation error. The sender must not close either of these streams, and the receiver must not request for their closure either. Closure of either unidirectional stream type must be treated as a connection error with the type being a "closed critical stream".

An endpoint may avoid creating an encoder stream if it will not be used for example, if the size of the dynamic table is zero or the encoder simply does not wish to use a dynamic table. For the decoder stream an endpoint may avoid creating it, if its decoder sets the maximum capacity of the dynamic table to zero. An endpoint must allow its peer to create an encoder stream and a decoder stream even if the connection's settings prevent their use.

Like HPACK, QPACK uses two tables, the dynamic and static tables for field lines. The static table for predefined common header field lines and the dynamic table for adding field lines during the connection. QPACK also defines unidirectional streams for sending instructions from the encoder to the decoder and vice versa [37].

QPACK's encoder converts a header or trailer section into a series of representations by giving either an indexed or a literal representation for each field line in the list. Indexed representations much like in HPACK achieve high compression by replacing either the literal name or the value or both with an index to either the static or dynamic table. References to the static table and literal representations never risk head-of-line blocking. References to the dynamic table risk head-of-line blocking if the encoder has not received an acknowledgment indicating the entry is available at the decoder. An encoder may insert any entry into the dynamic table it chooses and is not limited to the field lines it is compressing.

QPACK is designed to preserve the ordering of field lines within each field section. This means an encoder must emit field representations in the order they appear in the input field section. QPACK is also designed to place the burden of optional state tracking on the encoder, and this results in relatively simple decoders.

Inserting entries into the dynamic table might not be possible if the table contains entries that cannot be evicted. A dynamic table entry cannot be evicted immediately after insertion, even if it has never been referenced. Once the insertion of a dynamic table entry has been acknowledged and there are no outstanding references to the entry in unacknowledged representations, the entry becomes able to be evicted. References on the encoder stream never prevent the eviction of an entry, because those references are guaranteed to be processed before the instruction evicting the entry.

If the dynamic table does not contain enough room for a new entry without evicting other entries, and the entries that would be evicted are not able to be evicted, the encoder must not insert that entry into the dynamic table this includes duplicates of existing entries. In order to avoid this, the encoder that uses the dynamic table needs to keep track of each dynamic table entry, referenced by each field section until those representations are acknowledged by the decoder. Below in the table 9 is a simple example of how the dynamic table in QPACK works with insertion and dropping points.

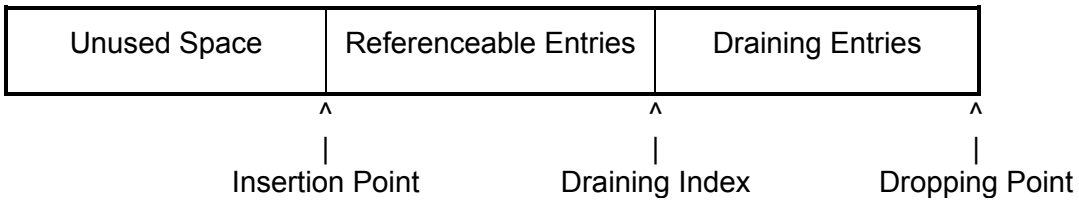

Table 9: QPACK Dynamic table insertion and dropping points [37]

To ensure that the encoder is not prevented from adding new entries, the encoder can avoid referencing entries that are close to eviction. Rather than reference such an entry, the encoder can emit a duplicate instruction and reference the duplicate instead.

Determining which entries are too close to eviction to reference is a preference of the encoder. One way is to target a fixed amount of available space in the dynamic table: either unused space or space that can be reclaimed by evicting non-blocking entries. To achieve this, the encoder can maintain a draining index, which is the smallest absolute index in the dynamic table that it will emit a reference for. This would in practice mean a dynamic table with the draining index within the larger dynamic table. As new entries are inserted, the encoder increases the draining index to maintain the section of the table that it will not reference. As the entries pass the draining index, the encoder does not create new references to them. This means that unacknowledged references to these entries with an absolute index lower than the draining index will eventually become zero, allowing them to be evicted. Also of note that as QPACK uses QUIC and it does not guarantee order between data on different streams, a decoder might encounter a representation that references a dynamic table entry that it has not yet received.

Each encoded field section contains a required insert count, which is the lowest possible value for the insert count with which the field section can be decoded. For a field section

encoded using references to the dynamic table, the required insert count is one larger than the largest absolute index of all referenced dynamic table entries. For a field section encoded with no references to the dynamic table, the required insert count is zero. When the decoder receives an encoded field section with a required insert count greater than its own insert count, the stream cannot be processed immediately and is considered blocked.

Compression efficiency can often be improved by referencing dynamic table entries that are still in transit, but if there is loss or reordering, the stream can become blocked at the decoder. An encoder can avoid the risk of blocking by only referencing dynamic table entries that have been acknowledged, but this could mean using literals. Since literals make the encoded field section larger, this can result in the encoder becoming blocked on congestion or flow-control limits. Writing instructions on streams that are limited by flow control can also produce deadlocks.

QPACK uses a flow-control credit system, which means the receiver (decoder) gives the sender (encoder) credits to be able to send streams and the sender needs to check if it has enough credits to send a stream to the receiver. A decoder might stop issuing flow-control credit on the stream that carries an encoded field section until the necessary updates are received on the encoder stream. This means stopping new flows of streams starting from the encoder. If the granting of flow-control credit on the encoder stream (or the connection as a whole) depends on the consumption and release of data on the stream carrying the encoded field section, a deadlock might occur.

Generally, a stream containing a large instruction can become deadlocked if the decoder withholds flow-control credit until the instruction is completely received. Thus, an encoder should not write an instruction unless enough stream and connection flow-control credit is available for the entire instruction.

QPACK has a known received count, which is the total number of dynamic table insertions and duplications acknowledged by the decoder. The encoder tracks the known received count in order to identify which dynamic table entries can be referenced without potentially blocking a stream. On the other side the decoder tracks the known received count in order to be able to send insert count increment instructions. A section acknowledgment instruction implies that the decoder has received all dynamic table state necessary to decode the field section. If the required insert count of the acknowledged field section is greater than the current known received count, the known received count is updated to that required insert count value.

Just as in HPACK, the decoder processes a series of representations and emits the corresponding field sections. It also processes instructions received on the encoder stream that modify the dynamic table. Note that encoded field sections and encoder stream instructions arrive on separate streams. This is unlike HPACK, where encoded field sections (header blocks) can contain instructions that modify the dynamic table, and there is no dedicated stream of HPACK instructions. On the other side the decoder must emit field lines in the order their representations appear in the encoded field section.

Upon receipt of an encoded field section, the decoder examines the required insert count. If the required insert count is less than or equal to the decoder's insert count, the field section can be processed right away. Otherwise, that field section's stream becomes blocked. While blocked, the encoded field section data should remain in the blocked stream's flow-control window. This data remains unusable until the stream becomes unblocked. Releasing the flow control prematurely would make the decoder vulnerable to memory exhaustion attacks. A stream becomes unblocked when the insert count becomes greater than or equal to the required insert count for all encoded field sections the decoder has started reading from that stream.

After the decoder finishes decoding a field section encoded using representations containing dynamic table references, it must emit a section acknowledgment instruction. A stream may carry multiple field sections in the case of intermediate responses, trailers, and pushed requests. The encoder interprets each section acknowledgment instruction in order starting from the earliest one sent, in order to keep track of field sections containing dynamic table references sent on the given stream. Meaning in a set of five field sections, the first acknowledgment from the decoder means the first field section sent and thus enabling to use the dynamic references for ones sent in that and continuing in that order first-in first-out.

When an endpoint receives a stream reset before the end of a stream or before all encoded field sections are processed on that stream, it generates a stream cancellation instruction. This also happens when it abandons the reading of a stream. This signals to the encoder that all references to the dynamic table on that stream are no longer valid. A decoder with a maximum dynamic table capacity equal to zero may omit sending stream cancellations, because the encoder cannot have any dynamic table references. An encoder also cannot infer from this instruction that any updates to the dynamic table have been received. The section acknowledgment and stream cancellation instructions permit the encoder to remove references to entries in the dynamic table. When an entry with an absolute index lower than the known received count has zero references, then it can be evicted.

After receiving new table entries on the encoder stream, the decoder chooses when to emit insert count increment instructions. Emitting this instruction after adding each new dynamic table entry will provide the timeliest feedback to the encoder but could also be redundant with other decoder feedback. By delaying an insert count increment instruction, the decoder might be able to gather multiple insert count increment instructions or replace them entirely with section acknowledgments. On the other hand, delaying too long may lead to compression inefficiencies if the encoder waits for an entry to be acknowledged before using it.

If the decoder encounters a reference in a field line representation to a dynamic table entry that has already been evicted or that has an absolute index greater than or equal to the declared required insert count it must treat this as a connection error. If the decoder encounters a reference in an encoder instruction to a dynamic table entry that has already been evicted, it must also treat this as a connection error.

Unlike in HPACK, entries in the QPACK static and dynamic tables are addressed separately. The static table consists of a predefined list of field lines, each of which has a fixed index over time. All entries in the static table have a name and a value, however, values can be empty. Each entry is identified by a unique index. Interestingly the QPACK static table is indexed from 0, whereas the HPACK static table is indexed from 1.

When the decoder encounters an invalid static table index in a field line representation, it must treat this as a decompression failure. If this index is received on the encoder stream, this must be treated as an encoder stream error.

The dynamic table consists of a list of field lines maintained in first-in, first-out order. A QPACK encoder and decoder share a dynamic table that is initially empty. The encoder adds entries to the dynamic table and sends them to the decoder via instructions on the encoder stream. Much like in HPACK the dynamic table can contain duplicate entries meaning entries with the same name and value, therefore duplicate entries must not be treated as an error by the decoder. Dynamic table entries can also have empty values.

The size of the dynamic table is the sum of the size of its entries. The size of an entry is the sum of its name's length in bytes, its value's length in bytes, and 32 additional bytes. The size of an entry is calculated using the length of its name and value without Huffman encoding applied. The encoder sets the capacity of the dynamic table, which serves as the upper limit on its size. The initial capacity of the dynamic table is zero. The encoder needs to send a "set dynamic table capacity" instruction, with a non-zero capacity to begin using the dynamic table.

Before a new entry is added to the dynamic table, entries are evicted from the end of the dynamic table until the size of the dynamic table is less than or equal to table capacity. Thus, the encoder must not cause a dynamic table entry to be evicted unless that entry is able to be evicted. If the encoder attempts to add an entry that is larger than the dynamic table capacity, the decoder must treat this as a connection error.

A new entry can reference an entry in the dynamic table that will be evicted when adding this new entry into the dynamic table. Implementations should not delete referenced names or values of an entry, if the referenced entry is evicted from the dynamic table prior to inserting the new entry. Whenever the dynamic table capacity is reduced by the encoder, entries are evicted from the end of the dynamic table until the size of the dynamic table is less than or equal to the new table capacity. This mechanism allows completely clearing the dynamic table by setting a capacity of 0, which can subsequently be restored.

To bound the memory requirements of the decoder, the decoder limits the maximum value the encoder is permitted to set for the dynamic table capacity. In HTTP/3, this limit is determined by the value sent by the decoder. The encoder must not set a dynamic table capacity that exceeds this maximum, but it can choose to use a lower dynamic table capacity. When the maximum table capacity is zero, the encoder must not insert entries into the dynamic table and must not send any encoder instructions on the encoder stream. Each entry possesses an absolute index that is fixed for the lifetime of that entry. The first entry inserted has an absolute index of 0, indices increase by one with each insertion. Relative indices begin at zero and increase in the opposite direction from the absolute index. Determining which entry has a relative index of 0 depends on the context of the reference. In the table below is an example for indexing in the encoder stream dynamic table, where 'n' is the amount of entries inserted and 'd' is the amount of entries dropped.

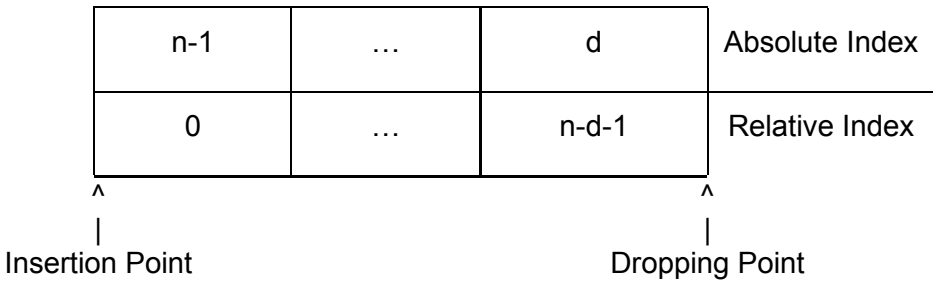

Table 10: Dynamic Table Indexing Encoder Stream [37]

In encoder instructions a relative index of 0 refers to the most recently inserted value in the dynamic table. Note that this means the entry referenced by a given relative index will change while interpreting instructions on the encoder stream. Unlike in encoder instructions, relative indices in field line representations are relative to the Base at the beginning of the encoded field section. This ensures that references are stable even if encoded field sections and dynamic table updates are processed out of order. Below is a visualization for dynamic table indexing with relative index in representation with Base = n-2. Otherwise same as table 10, 'n' being the amount of entries inserted and 'd' being the amount of entries dropped.

		Base ↓			
n-1	n-2	n-3	…	d	Absolute Index
		0	…	n-d-3	Relative Index

Table 11: Dynamic table indexing – relative index in representation [37]

Post-Base indices are used in field line representations for entries with absolute indices greater than or equal to Base, starting at 0 for the entry with absolute index equal to Base and increasing in the same direction as the absolute index. Post-Base indices allow an encoder to process a field section in a single pass and include references to entries added while processing this field section or other field sections. For this table 12 below shows an example for dynamic table indexing with post-base index in representation, with the base being equal to n-2 as before in table 11. Also, same as before 'n' being the amount of entries inserted and 'd' being the amount of entries dropped.

		Base ↓			
n-1	n-2	n-3	…	d	Absolute Index
1	0				Post-Base Index

Table 12: Dynamic table indexing – Post-Base Index in representation. [37]

Next are more in-depth looks on how exactly the flow of an encoder work. So, an encoder sends encoder instructions on the encoder stream to set the capacity of the dynamic table and add dynamic table entries. Instructions adding table entries can use existing entries to avoid transmitting redundant information. The name can be transmitted as a reference to an existing entry in the static or the dynamic table or as a string literal. For entries that already exist in the dynamic table, the full entry can also be used by reference, creating a duplicate entry.

More specifically an encoder informs the decoder of a change to the dynamic table capacity using an instruction that starts with the '001' 3-bit pattern. This is followed by the new dynamic table capacity represented as an integer with a 5-bit prefix.

The new capacity must be lower than or equal to the limit. In HTTP/3, this limit is the value of the "QPACK max table capacity" parameter received from the decoder. The decoder must treat a new dynamic table capacity value that exceeds this limit as a connection error of type "QPACK encoder stream error". As said before reducing the dynamic table capacity can cause entries to be evicted. This must not cause the eviction of entries that are not able to be evicted. Changing the capacity of the dynamic table is not acknowledged as this instruction does not insert an entry. Below is an example of field line insertion with an indexed name.

0	1	2	3	4	5	6	7
1	T	Name Index (6+)					
H	Value Length (7+)						
Value String (Length bytes)							

Table 13: Insert Field Line – Indexed Name [37]

Concerning the table above, an encoder adds an entry to the dynamic table where the field name matches the field name of an entry stored in the static or the dynamic table using an instruction that starts with the '1' 1-bit pattern. The second 'T' bit indicates whether the reference is to the static or dynamic table. The 6-bit prefix integer that follows is used to locate the table entry for the field name. When T=1, the number represents

the static table index, when T=0, the number is the relative index of the entry in the dynamic table. Also, as before in HPACK examples if 'H' is 0 then the encoded data is the raw octets, if 'H' is 1 then the encoded data is the Huffman encoding of the string.

0	1	2	3	4	5	6	7
0	1	H	Name Length (5+)				
Name String (Length bytes)							
H	Value Length (7+)						
Value String (Length bytes)							

Table 14: Insert Field Line – new name [37]

In the above table 14 is an example for inserting a field line with a new name. In this case an encoder adds an entry to the dynamic table where both the field name and the field value are represented as string literals. This uses an instruction that starts with the '01' 2-bit pattern. This is followed by the name represented as a 6-bit prefix string literal and the value represented as an 8-bit prefix string literal.

For duplicating and existing entry in the dynamic table, the encoder uses an instruction that starts with the '000' 3-bit pattern. This is followed by the relative index of the existing entry represented as an integer with a 5-bit prefix. The existing entry is reinserted into the dynamic table without resending either the name or the value. This is useful to avoid adding a reference to an older entry, which might block inserting new entries.

After processing an encoded field section whose declared required insert count is not zero, the decoder emits a section acknowledgment instruction. The instruction starts with the '1' 1-bit pattern, followed by the field section's associated stream ID encoded as a 7-bit prefix integer. If an encoder receives a section acknowledgment instruction referring to a stream on which every encoded field section with a non-zero required insert count has already been acknowledged. This is to be treated as a connection error of type "QPACK decoder stream error". When a stream is reset or reading is abandoned, the decoder emits a stream cancellation instruction. The instruction starts with the '01' 2-bit pattern, followed by the stream ID of the affected stream encoded as a 6-bit prefix integer.

The insert count increment instruction starts with the '00' 2-bit pattern, followed by the Increment encoded as a 6-bit prefix integer. This instruction increases the known received count by the value of the increment parameter. The decoder should send an increment value that increases the known received count to the total number of dynamic table insertions and duplications processed so far. An encoder that receives an increment field equal to zero or one that increases the known received count beyond what the encoder has sent, must be treated as a connection error.

An encoded field section consists of a prefix and a possibly empty sequence of representations. Each representation corresponds to a single field line. These representations reference the static table or the dynamic table in a state, but they do not modify that state. Encoded field sections are carried in frames on streams defined by the enclosing protocol. Each encoded field section is prefixed with two integers. The required insert count is encoded as an integer with an 8-bit prefix using the encoding. The base is encoded as a sign bit ('S') and a delta base value with a 7-bit prefix. An example below in table 15 shows how the encoded field section works.

0	1	2	3	4	5	6	7
Required Insert Count (8+)							
S	Delta Base (7+)						
Encoded Field Lines							

Table 15: Encoded Field Section [37]

Required insert count identifies the state of the dynamic table needed to process the encoded field section. Blocking decoders use the required insert count to determine when it is safe to process the rest of the field section. This encoding limits the length of the prefix on long-lived connections. The base is used to resolve references in the dynamic table. The base is encoded relative to the required insert count using a one-bit sign and the delta base value. A sign bit of 0 would indicate that the base is greater than or equal to the value of the required insert count. The decoder adds the value of the delta base to the required insert count to determine the value of the base. If the sign bit is 1,

then that indicates that the base is less than the required insert count. the decoder subtracts the value of delta base from the required insert count and also subtracts one to determine the value of the base.

A single-pass encoder determines the base before encoding a field section. In a scenario where the encoder inserted entries in the dynamic table while encoding the field section and references them. It would mean that the required insert count will be greater than the base, so the encoded difference is negative, and the sign bit is set to 1. If the field section was not encoded using representations that reference the most recent entry in the table and did not insert any new entries, the base will be greater than the required insert count. This means the encoded difference will be positive and the sign bit is set to 0.

The value of base must not be negative. Though the protocol might operate correctly with a negative base using post-base indexing, it is unnecessary and inefficient. An endpoint must treat a field block with a sign bit of 1 as invalid if the value of required insert count is less than or equal to the value of delta base. An encoder that produces table updates before encoding a field section might set base to the value of required insert count. In such a case, both the sign bit and the delta base will be set to zero.

A field section that was encoded without references to the dynamic table can use any value for the base. Setting the delta base to zero is one of the most efficient encodings. For example, with a required insert count of 9, a decoder receives a sign bit of 1 and a delta base of 2. This sets the base to 6 and enables post-base indexing for three entries. in this example, a relative index of 1 refers to the fifth entry that was added to the table, a post-base index of 1 refers to the eighth entry. An indexed field line with post-base index representation identifies an entry in the dynamic table with an absolute index greater than or equal to the value of the base.

Now for field representations. A literal field line with name reference representation encodes a field line where the field name matches the field name of an entry in either the static or dynamic table with an absolute index less than the value of the base. Table 16 below shows an example of how this works.

0	1	2	3	4	5	6	7
0	1	N	T	Name Index (4+)			
H	Value Length (7+)						
Value String (Length bytes)							

Table 16: Literal field line with name reference [37]

This representation starts with the '01' 2-bit pattern. The following bit, 'N', indicates whether an intermediary is permitted to add this field line to the dynamic table on subsequent hops. When the 'N' bit is set, the encoded field line must always be encoded with a literal representation. When a peer sends a field line that it received represented as a literal field line with the 'N' bit set, it must use a literal representation to forward this field line. This bit is intended for protecting field values that are not to be put at risk by compressing them. The fourth ('T') bit indicates whether the reference is to the static or dynamic table. The 4-bit prefix integer that follows is used to locate the table entry for the field name. When T=1, the number represents the static table index, when T=0, the number is the relative index of the entry in the dynamic table.

A literal field line with post-base name reference representation encodes a field line where the field name matches the field name of a dynamic table entry with an absolute index greater than or equal to the value of the base. This representation starts with the '0000' 4-bit pattern. The fifth bit is the 'N' bit, this is followed by a post-Base index of the dynamic table entry encoded as an integer with a 3-bit prefix. Only the field name is taken from the dynamic table entry, the field value is then encoded as an 8-bit prefix string literal.

The literal field line with literal name representation encodes a field name and a field value as string literals. Table 17 below shows the visual example of how this works in practice.

0	1	2	3	4	5	6	7
0	0	1	N	H	Name Length (3+)		
Name String (Length bytes)							
H	Value Length (7+)						
Value String (Length bytes)							

Table 17: Literal field line with literal name [37]

This representation starts with the '001' 3-bit pattern. The fourth bit is the 'N' bit. The name follows, represented as a 4-bit prefix string literal, then the value, represented as an 8-bit prefix string literal.

Lastly the potential areas of security concerns with QPACK. These include ones such as using compression as a length-based oracle for verifying guesses about secrets that are compressed into a shared compression context or denial of service resulting from exhausting processing or memory capacity at a decoder

Probing the dynamic table state should also be taken into consideration as QPACK reduces the encoded size of field sections by exploiting the redundancy inherent in protocols like HTTP. The goal of this is to reduce the amount of data that is required to send HTTP requests or responses. Much like in HPACK the compression context used to encode header and trailer fields can be probed by an attacker who can both define fields to be encoded and transmitted and observe the length of those fields once they are encoded. When an attacker can do both, they can adaptively modify requests in order to confirm guesses about the dynamic table state. If a guess is compressed into a shorter length, the attacker can observe the encoded length and infer that the guess was correct.

This is possible even over the TLS and QUIC because while TLS and QUIC provide confidentiality protection for content, they only provide a limited amount of protection for the length of that content. Same as in HPACK padding schemes only provide limited protection against an attacker with these capabilities, potentially only forcing an increased number of guesses to learn the length associated with a given guess. Padding schemes also work against compression by increasing the number of bits that are transmitted.

Same as in HPACK CRIME attacks work similarly with QPACK as it mitigates but does not completely prevent attacks modelled on CRIME by forcing a guess to match an entire field line rather than individual characters. An attacker can only learn whether a guess is correct or not, so the attacker is reduced to a brute-force guess for the field values associated with a given field name. So once again the viability of recovering specific field values depends on the entropy of values. As a result, values with high entropy are unlikely to be recovered successfully. However, values with low entropy remain vulnerable. Much like in HPACK, attacks of this nature are possible any time that two mutually distrustful entities control requests or responses that are placed onto a single HTTP/3 connection. If the shared QPACK compressor permits one entity to add entries to the dynamic table, and the other to refer to those entries while encoding chosen field lines, then the attacker can learn the state of the table by observing the length of the encoded output.

For example, requests or responses from mutually distrustful entities can occur when an intermediary either sends requests from multiple clients on a single connection toward an origin server or takes responses from multiple origin servers and places them on a shared connection toward a client. Web browsers also need to assume that requests made on the same connection by different web origins are made by mutually distrustful entities. Other scenarios involving mutually distrustful entities are also possible. Users of HTTP that require confidentiality for header or trailer fields face the same choice as with HPACK, sufficient entropy may be achieved but is impractical to use due to added constraints.

Same as with HPACK rather than impose constraints on users of HTTP, an implementation of QPACK can instead constrain how compression is applied in order to limit the potential for dynamic table probing. An ideal solution segregates access to the dynamic table based on the entity that is constructing the message. Field values that are added to the table are attributed to an entity, and only the entity that created a value can extract that value. And as well to improve compression performance of this option, certain entries might be tagged as being public. Like with HPACK, web browser could once again make the values of certain headers available in all requests.

Also just like with HPACK an encoder might introduce a penalty for many field lines with the same field name and different values. This penalty could cause many attempts to guess a field value to result in the field not being compared to the dynamic table entries in future messages, effectively preventing further guesses. This response might be made inversely proportional to the length of the field value. Disabling access to the dynamic table for a given field name might occur for shorter values more quickly or with higher

probability than for longer values. This mitigation is most effective between two endpoints. If messages are re-encoded by an intermediary without knowledge of which entity constructed a given message, the intermediary could inadvertently merge compression contexts that the original encoder had specifically kept separate.

Once again in similar vein to HPACK, simply removing entries corresponding to the field from the dynamic table can be ineffectual if the attacker has a reliable way of causing values to be reinstalled. Implementations can also choose to protect sensitive fields by not compressing them and instead encoding their value as literals. Refusing to insert a field line into the dynamic table is only effective if doing so is avoided on all hops. The never-indexed literal bit can be used to signal to intermediaries that a value was intentionally sent as a literal.

An intermediary must not re-encode a value that uses a literal representation with the 'N' bit set with another representation that would index it. If QPACK is used for re-encoding, a literal representation with the 'N' bit set must be used. If HPACK is used for re-encoding, the never-indexed literal representation must be used.

The choice to mark that a field value should never be indexed depends on several factors. Since QPACK does not protect against guessing an entire field value, short or low-entropy values are more readily recovered by an adversary. Therefore, an encoder might choose not to index values with low entropy. An encoder might also choose not to index values for fields that are highly valuable or sensitive to recovery, such as the Cookie or Authorization header fields.

And the same as was with HPACK, an encoder might prefer indexing values for fields that have little or no value if they were exposed. Also noting that these criteria for deciding to use a never-indexed literal representation will evolve over time as new attacks are discovered.

As said before with HPACK, there is no currently known attack against a static Huffman encoding. A study[36] has shown that using a static Huffman encoding table created an information leakage, however, this same study concluded that an attacker could not take advantage of this information leakage to recover any meaningful amount of information.

An attacker can try to cause an endpoint to exhaust its memory. QPACK is designed to limit both the peak and stable amounts of memory allocated by an endpoint. QPACK uses the definition of the maximum size of the dynamic table and the maximum number of blocking streams to limit the amount of memory the encoder can cause the decoder to consume. The limit on the size of the dynamic table considers the size of the data stored in the dynamic table, plus a small overhead. The limit on the number of blocked

streams is only a proxy for the maximum amount of memory required by the decoder. The actual maximum amount of memory will depend on how much memory the decoder uses to track each blocked stream.

A decoder can limit the amount of state memory used for the dynamic table by setting an appropriate value for the maximum size of the dynamic table. An encoder can limit the amount of state memory it uses by choosing a smaller dynamic table size than the decoder allows and signalling this to the decoder. A decoder can also limit the amount of state memory used for blocked streams by setting an appropriate value for the maximum number of blocked streams. Streams that risk becoming blocked consume no additional state memory on the encoder.

An encoder allocates memory to track all dynamic table references in unacknowledged field sections. An implementation can directly limit the amount of state memory by only using as many references to the dynamic table as it wishes to track, no signalling to the decoder is required. Limiting references to the dynamic table does reduce compression effectiveness.

The amount of temporary memory consumed by an encoder or decoder can be limited by processing field lines sequentially. A decoder implementation does not need to retain a complete list of field lines while decoding a field section. An encoder implementation does not need to retain a complete list of field lines while encoding a field section if it is using a single-pass algorithm. It might be necessary for an application to retain a complete list of field lines for other reasons, even if QPACK does not force this to occur.

While the negotiated limit on the dynamic table size accounts for most of the memory that can be consumed by a QPACK implementation. Data that cannot be immediately sent due to flow control is not affected by this limit. Thus, implementations should limit the size of unsent data, especially on the decoder stream where flexibility to choose what to send is limited. Possible responses to an excess of unsent data include limiting the ability of the peer to open new streams or reading only from the encoder stream or simply closing the connection.

An implementation of QPACK needs to ensure that large values for integers, long encoding for integers, or long string literals do not create security weaknesses. An implementation must set a limit for the values it accepts for integers, as well as for the encoded length. In the same way, it must set a limit to the length it accepts for string literals. These limits should be large enough to process the largest individual field the HTTP implementation can be configured to accept.

If an implementation encounters a value larger than it can decode, this must be treated as a decompression failed stream error if on a request stream or a connection error of the appropriate type if on the encoder or decoder stream. So, while there are many considerations when it comes to security and QPACK, there are ways to mitigate security risks with how an implementation of QPACK is made.

4.4 Performance and results

This section covers studies both general and in-depth on the performance of implementations of compression algorithms themselves as well as the HTTP versions to give a general look into the performance side of HTTP and its headers.

One study by Kazuhiko Yamamoto, Tatsuhiro Tsujikawa and Kazuho Oku tested HPACK header compression in their study " Exploring HTTP/2 Header Compression "[38]. Their sample data consisted of 31 sets, where each set had multiple headers anywhere from 2 to 646. They tested 8 compression methods of HPACK and found them generally effective on both the static and dynamic tables and Huffman encoding. In their conclusion they state "HPACK encoding with our techniques is 2.10x faster than the naive implementation. Also,our 4-bits based HPACK decoding is 2.69x faster than the original bit-by-bit implementation."[38]

Another study evaluated HTTP versions based on how they reacted to latency and packet loss as well as the general usage. In their evaluation part of the study "A Brief Overview on HTTP" [39] by Justus Wendroth and Benedikt Jaeger a comparison was made on the effects of latency. They concluded in terms of handshakes done HTTP/3 performs the best as the cryptographic and transport handshakes are combined. Whereas HTTP/2 and HTTP/1.1 need more handshakes between the client and server to accomplish the connection. Also mentioned is that an increase in latency would affect HTTP/3 the least with HTTP/2 being second and most affected would be HTTP/1.1. Also mentioned in their evaluation is that in some cases HTTP/3 can perform worse than HTTP/2. This happened when comparing latency in a test measuring time for the largest payload to be rendered completely. In their study the effects of packet loss were most severe for HTTP/2 due to the head of line blocking and as HTTP/3 solves this part by using QUIC and streams of data. It was then concluded that HTTP/3 performed best in terms of packet loss with HTTP/2 and HTTP/1.1 having mixed results on which was better than the other. At the end the conclusion was "h2 performs better than h1 at high latency. For higher packet loss, the benefits of h2 are reduced and h1 can also perform better. By comparing h3 and h2, we see that h3 performs better under higher packet

loss. For higher latency, h2 performs better for some tests and h3 for others [39], they used "h1" for HTTP/1.1 and "h2" for HTTP/2 and "h3" for HTTP/3.

One study comparing HTTP/1.1 and HTTP/2 the "Client-side Energy Efficiency of HTTP/2 for Web and Mobile App Developers"[40] by Shaiful Alam Chowdhury, Varun Sapra and Abram Hindle, uses power and energy as the measurement for comparing the HTTP versions. Their testing lead to conclusions that compared to HTTP/1.1, HTTP/2 does save energy, when round trip times are above 30ms and when TLS is being used. The tests also indicated that HTTP/2 outperformed HTTP/1.1 with TLS in most scenarios. In their study they also mention " The advantage of HTTP/2 highly depends on the round-trip time between the client and the server. HTTP/1.1 becomes expensive, for large number of TCP connections, with large number of objects. This becomes even worse when the RTT is higher, which is common in cellular data networks. HTTP/2, on the other hand, does not have this problem as it deals with one single connection by incorporating a multiplexing technique."[40]. Also, an interesting side mention in their work is the fact that they observed that accessing the internet has become more energy expensive after the introduction of HTTPS, which does make sense as adding complexity will require more computing power. Their conclusions were that the web server over HTTP/2 is more mobile user friendly especially on high-latency wireless and Wi-Fi networks. Though in their study they also mention that they did not consider the energy consumption on the server-side.

In the study "Performance Evaluation and Improvement of HTTP/3 Considering TLS"[41] by Ryuichi Niitsu and Saneyasu Yamaguchi, HTTP/3 and HTTP/1.1 was compared in terms of file transfer. In their study they found the performance of HTTP/3 being remarkably less than HTTP/1.1 and that the performance of HTTP/3 decreased as the packet loss increased. In the study they discussed that the performance might have been due to them testing with the default MSS (Maximum Segment Size) of 1252 and in HTTP/1.1 it is automatically set to the largest size by the operating system. In the case with HTTP/3 and QUIC it was discovered that MSS is determined by a userland process and as a result an optimal size, which would be in most cases the largest was not chosen. They concluded that HTTP/3 performance could be improved by increasing the MSS of HTTP/3 and that the default MSS is pessimistically set. Another important point in their study was the testing of HTTP/1.1 and HTTP/3 with the same version of TLS, that being TLS 1.3, which is normally used in HTTP/3 and even then, HTTP/1.1 remarkably outperformed HTTP/3. At the end of their study the conclusion was " We then showed that, in most cases, the performance of HTTP/3 is inferior to that of HTTP/1.1. We then discussed the reasons for the lower performance and explored the relationship between

MSS and HTTP/3 performance, showing that increasing MSS improves HTTP/3 performance."[41]

Lastly of note is a survey published in IEEE communications surveys & tutorials, the "Packet Header Compression: A Principle-Based Survey of Standards and Recent Research Studies"[42] by Máté Tömösközi, Martin Reisslein, Frank H. P. Fitzek. In their article they provide a comprehensive survey of major packet header compression standards and packet header compression research studies recent to the surveys time in 2022. In the survey there is also important general level and in-depth information on packet header compressions and why they are used. In their survey are a few mentions especially relevant to this study. One being a mentioning of QPACK with the following line "Due to its novelty, at the time of writing, there are no publicly available evaluations of QPACK"[42], which this study also corroborates, as is evident in this section with performance and results, with a lack of result for QPACK itself. As seemingly QPACK still is an emerging technology the publicly available results for its efficiency are yet to be easily found. Otherwise the survey sheds light into the many possibilities in header compression standards today and gives insight as to what it may look in the future, with mentions of Static Context Header Compression (SCHC) and combining header compression with machine learning and coding to create more efficient models especially in complex environments.

5. CONCLUSIONS

This study went over how HTTP has evolved from what it started, that being later named the HTTP/0.9 and the iterations of it from HTTP/1.0 to HTTP/1.1, the later versions of HTTP/2 and the latest HTTP/3 were also delved into. The differences of each version of HTTP was discussed with special attention to how HTTP/3 works with QUIC and its differences to the TCP pipeline that is used on the prior versions of HTTP. It was also mentioned that the different versions of HTTP don't obsolete each other by design and are used by how well they serve the intended purpose. Next this study delved into the headers of HTTP and discussed what they were, how they are used and showed examples. There were many types of categories of headers from general categories to the more specific ones based on the header use case. It was also discussed that headers are ever evolving as some headers work with multiple versions of HTTP and some do not. Some headers might not work with newer versions of HTTP and are obsoleted for them and new headers might need to be introduced to replace them, this means that there are headers unique to each version of HTTP as well as general headers used by all.

This study also went deeper into the compression algorithms used on the headers of HTTP, with the introduction of HPACK in HTTP/2 and QPACK in HTTP/3. The implementation dynamics and differences of operation between HPACK and QPACK was discussed alongside security considerations for each. At the end of this study the general performance of each version of HTTP was discussed alongside a look into the efficiency of compression algorithms. Many different studies were taken into account and the results were that generally each version of HTTP was an improvement from the last with HTTP/3 generally being the best performer. Yet some studies also proved there are scenarios where HTTP/2 can outperform HTTP/3 reinforcing the fact that different versions of HTTP do not obsolete each other even in a performance case. Studies in HPACK also showed great results for the compression algorithm but any publicly available studies of QPACK's performance was not found and thus comparisons could not be made between the compression algorithms. The absence of studies into QPACK's efficiency was discussed to be most likely due to the fact that QPACK is quite a new technology.

REFERENCES

[1] Borenstein, N. and N. Freed, "MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies", RFC 1521, DOI 10.17487/RFC1521, September 1993, https://www.rfc-editor.org/info/rfc1521

[2] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, https://www.rfc-editor.org/info/rfc9110

[3] Berners-Lee, T., Fielding, R., and H. Frystyk, "Hypertext Transfer Protocol -- HTTP/1.0", RFC 1945, DOI 10.17487/RFC1945, May 1996, https://www.rfc-editor.org/info/rfc1945

[4] Mogul, J., Fielding, R., Gettys, J., and H. Frystyk, "Use and Interpretation of HTTP Version Numbers", RFC 2145, DOI 10.17487/RFC2145, May 1997, https://www.rfc-editor.org/info/rfc2145

[5] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, DOI 10.17487/RFC2616, June 1999, https://www.rfc-editor.org/info/rfc2616

[6] Belshe, M., Peon, R., and M. Thomson, Ed., "Hypertext Transfer Protocol Version 2 (HTTP/2)", RFC 7540, DOI 10.17487/RFC7540, May 2015, https://www.rfc-editor.org/info/rfc7540

[7] Rescorla, E., "HTTP Over TLS", RFC 2818, DOI 10.17487/RFC2818, May 2000, https://www.rfc-editor.org/info/rfc2818

[8] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, https://www.rfc-editor.org/info/rfc8446

[9] Postel, J., "Internet Protocol", STD 5, RFC 791, DOI 10.17487/RFC0791, September 1981, https://www.rfc-editor.org/info/rfc791

[10] Eddy, W., Ed., "Transmission Control Protocol (TCP)", STD 7, RFC 9293, DOI 10.17487/RFC9293, August 2022, https://www.rfc-editor.org/info/rfc9293

[11] Bishop, M., Ed., "HTTP/3", RFC 9114, DOI 10.17487/RFC9114, June 2022, https://www.rfc-editor.org/info/rfc9114

[12] Iyengar, J., Ed., and M. Thomson, Ed., "QUIC: A UDP-Based Multiplexed and Secure Transport", RFC 9000, DOI 10.17487/RFC9000, May 2021, https://www.rfc-editor.org/info/rfc9000

[13] Roland Zegers. HTTP Header Analysis 2015. https://www.os3.nl/_media/2014-2015/courses/rp2/p91_report.pdf

[14] QUIC at 10,000 feet. https://docs.google.com/document/d/1gY9-YNDNAB1eip-RTPbqphgySwSNSDHLq9D5Bty4FSU

[15] Péter Megyesi, Zsolt Krämer, Sándor Molnár. How quick is QUIC? 2016 IEEE International Conference on Communications (ICC), Kuala Lumpur, Malaysia, 2016, pp. 1-6, doi: 10.1109/ICC.2016.7510788.

[16] J. Roskind, “QUIC - Multiplexed Stream Transport over UDP.” http://www.ietf.org/proceedings/88/slides/slides-88-tsvarea-10.pdf

[17] Quentin De Coninck and Olivier Bonaventure. Multipath QUIC: Design and Evaluation. 2017. In Proceedings of CoNEXT ’17: The 13th International Conference on emerging Networking EXperiments and Technologies, Incheon Republic of Korea, December 12–15, 2017 (CoNEXT ’17), 7 pages, https://doi.org/10.1145/3143361.3143370

[18] Tobias Viernickel, Alexander Frömmgen, Amr Rizk, Boris Koldehofe, Ralf Steinmetz Multipath QUIC: A Deployable Multipath Transport Protocol. 2018. IEEE International Conference on Communications (ICC), Kansas City, MO, USA, 2018. pp. 1-7, doi: 10.1109/ICC.2018.8422951.

[19] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira. PCC: Re-architecting congestion control for consistent high performance. 2015. In Proc. of USENIX NSDI. https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-dong.pdf

[20] N. Dukkipati, N. Cardwell, Y. Cheng, and M. Mathis. Tail Loss Probe (TLP): An Algorithm for Fast Recovery of Tail Losses. 2013. https://tools.ietf.org/html/draft-dukkipati-tcpm-tcp-loss-probe-01

[21] M. Mathis, N. Dukkipati, and Y. Cheng. Proportional rate reduction for TCP. 2013. https://tools.ietf.org/html/rfc6937

[22] J. Roskind. 2012. QUIC: Design Document and Specification Rationale. (2012). https://goo.gl/eCYF1a

[23] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-The Chang, Zhongyi Shi . 2017. The QUIC Transport Protocol: Design and Internet-Scale Deployment. In Proceedings of SIGCOMM ’17, Los Angeles, CA, USA, August 21-25, 2017, 14 pages. https://doi.org/10.1145/3098822.3098842

[24] https://www.catchpoint.com/http2-vs-http3/quic-vs-tcp

[25] https://shurutech.com/the-evolution-of-http

[26] M. Fischlin and F. Günther. Multi-stage key exchange and the case of Google's QUIC protocol. 2014 In Proc. of ACM CCS '14. https://doi.org/10.1145/2660267.2660308

[27] Tibor Jager, Jörg Schwenk, and Juraj Somorovsky. 2015. On the Security of TLS 1.3 and QUIC Against Weaknesses in PKCS#1 v1.5 Encryption. In Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15). Association for Computing Machinery, New York, NY, USA, 1185?1196. https://doi.org/10.1145/2810103.2813657

[28] R. Lychev, S. Jero, A. Boldyreva and C. Nita-Rotaru, "How Secure and Quick is QUIC? Provable Security and Performance Analyses," 2015 IEEE Symposium on Security and Privacy, San Jose, CA, USA, 2015, pp. 214-231, doi: 10.1109/SP.2015.21.

[29] Y. Yu, M. Xu and Y. Yang, "When QUIC meets TCP: An experimental study," 2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC), San Diego, CA, USA, 2017, pp. 1-8, doi: 10.1109/PCCC.2017.8280429.

[30] M. Seufert, R. Schatz, N. Wehner and P. Casas, "QUICker or not? -an Empirical Analysis of QUIC vs TCP for Video Streaming QoE Provisioning," 2019 22nd Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN), Paris, France, 2019, pp. 7-12, doi: 10.1109/ICIN.2019.8685913.

[31] S. Cook, B. Mathieu, P. Truong and I. Hamchaoui, "QUIC: Better for what and for whom?" 2017 IEEE International Conference on Communications (ICC), Paris, France, 2017, pp. 1-6, doi: 10.1109/ICC.2017.7997281.

[32] https://http.dev/headers

[33] https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

[34] https://wireshark.org

[35] Peon, R. and H. Ruellan, "HPACK: Header Compression for HTTP/2", RFC 7541, DOI 10.17487/RFC7541, May 2015, https://www.rfc-editor.org/info/rfc7541

[36] Jiaqi Tan, Jayvardhan Nahata 2013. PETAL: Preset Encoding Table Information Leakage. https://www.pdl.cmu.edu/PDL-FTP/associated/CMU-PDL-13-106.pdf

[37] Krasic, C., Bishop, M., and A. Frindell, Ed., "QPACK: Field Compression for HTTP/3", RFC 9204, DOI 10.17487/RFC9204, June 2022, https://www.rfc-editor.org/info/rfc9204

[38] Kazuhiko Yamamoto, Tatsuhiro Tsujikawa and Kazuho Oku. 2017. Exploring HTTP/2 Header Compression. In *Proceedings of CFI'17, Fukuoka, Japan, June 14-16, 2017, 5 pages.* https://doi.org/10.1145/3095786.3095787

[39] Justus Wendroth, Benedikt Jaeger 2022. A Brief Overview on HTTP. in Proceedings of the seminar Innovative Internet Technologies and Mobile Communications (IITM) Summer Semester 2022. https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2022-11-1/NET-2022-11-1_11.pdf

[40] Shaiful Alam Chowdhury, Varun Sapra, Abram Hindle. Client-side Energy Efficiency of HTTP/2 for Web and Mobile App Developers. https://softwareprocess.es/pubs/chowdhury2016SANER-http2.pdf

[41] Ryuichi Niitsu, Saneyasu Yamaguchi 2023 Performance Evaluation and Improvement of HTTP/3 Considering TLS. in Proceedings of 2023 Eleventh International Symposium on Computing and Networking Workshops (CANDARW). https://www.cs.hiroshima-u.ac.jp/Proceedings23/CANDAR%202023/pdfs/CANDARW2023-dDGdIIHvWRXlB0gY3qHj3/069400a368/069400a368.pdf

[42] Máté Tömösközi, Martin Reisslein, Frank H. P. Fitzek. Packet Header Compression: A Principle-Based Survey of Standards and Recent Research Studies in: IEEE Communications Surveys & Tutorials (Volume: 24, Issue: 1, Firstquarter 2022) pp.698-740 https://ieeexplore.ieee.org/document/9686051